

## **UNIT II: CRYPTOGRAPHIC ALGORITHMS (12 HRS)**

2.1. Classical Cryptosystems: Caesar, Vigenère, Playfair, Rail Fence Ciphers

2.2. Modern Ciphers: Block vs. Stream Ciphers, Symmetric vs. Asymmetric Ciphers

2.3. Symmetric Encryption: Feistel Cipher Structure, DES, Basic Concepts of Fields (Groups, Rings, Fields, Modular Arithmetic, Galois Fields, Polynomial Arithmetic), and AES

2.4. Number Theory: Prime Numbers, Fermat's Theorem, Miller-Rabin Algorithm, Euclidean and Extended Euclidean Theorem, Euler Totient Function

2.5. Asymmetric Encryption: Diffie-Hellman Key Exchange, RSA Algorithm

## 2.1 CLASSICAL CRYPTOSYSTEMS

- Classical cryptosystems are the foundation of cryptography, primarily used before the advent of modern computers.
- They rely on two main techniques: **Substitution** (replacing characters) and **Transposition** (rearranging characters).
  1. Caesar
  2. Vigenère,
  3. Playfair,
  4. Rail Fence Ciphers

# CAESAR CIPHER

- The **Caesar Cipher** is a substitution cipher named after Julius Caesar, who used it for military communication.
  - In Caesar Cipher each letter in the plaintext is shifted by a fixed number of positions in the alphabet.
  - Example: If we encrypt **HELLO** with shift 3:  
H → K, E → H, L → O, L → O, O → R
  - Encrypted text: **KHOOR**
- ◆ **Formula:**  $C = (P + k) \bmod 26$

Where:

P = Plaintext letter position

k = Shift value

C = Ciphertext letter

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 |
| A  | B  | C  | D  | E  | F  | G  | H  | I  | J  | K  | L  | M  |
| 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| N  | O  | P  | Q  | R  | S  | T  | U  | V  | W  | X  | Y  | Z  |

# CAESAR CIPHER

## ■ Decryption Formula:

$$P = (C - k) \bmod 26$$

- If the result is negative:  $x \bmod 26 = x + 26$

## Advantages:

- i. It is very simple to understand and implement.
- ii. It requires only a small key (shift value).
- iii. It can be done easily without computers.

## Disadvantages:

- i. It is very easy to break using brute force (only 25 possible keys).
- ii. Letter frequency remains unchanged, so frequency analysis can break it quickly.
- iii. It provides very low security.

# VIGENÈRE CIPHER

- The **Vigenère Cipher** is a polyalphabetic substitution cipher developed by Giovan Battista Bellaso.

## How it works:

- Uses a **keyword** to determine the shift for each letter.
- Each letter of the keyword represents a shift value.

**Example:** Plaintext: H E L L O  
Keyword: K E Y K E

- Using letter positions:  
H + K, E + E, L + Y, L + K, O + E
- Encrypted text: **RIJVS**

Encryption formula:

$$C = (P + K) \bmod 26$$

Where:

P = Plaintext value

K = Key value

C = Ciphertext value

# VIGENÈRE CIPHER

- **Decryption Formula:**  $P = (C - K) \bmod 26$
- If the result is negative:  
$$x \bmod 26 = x + 26$$

## Advantages

- It is more secure than the Caesar cipher because it uses multiple shifts.
- It reduces the effectiveness of simple frequency analysis.
- It is still relatively easy to implement manually.

## Disadvantages:

- It can be broken using methods like frequency analysis.
- If the key is short or repeated, security becomes weak.
- Key management can be difficult.

## Example

Ciphertext: RIJVS

Key: KEY

$$R - K = H$$

$$I - E = E$$

$$J - Y = L$$

$$V - K = L$$

$$S - E = O$$

Plaintext: **HELLO**

# PLAYFAIR CIPHER

- The **Playfair Cipher** is a digraph substitution cipher invented by Charles Wheatstone and promoted by Lord Playfair.

## How it works:

- Encrypts **pairs of letters (digraphs)** instead of single letters.
- A 5×5 matrix is created using a keyword.
- I and J are usually combined.

|   |   |   |     |   |
|---|---|---|-----|---|
| M | O | N | A   | R |
| C | H | Y | B   | D |
| E | F | G | I/J | K |
| L | P | Q | S   | T |
| U | V | W | X   | Z |

## Rules:

- Same row → Replace with letters to the right.
- Same column → Replace with letters below.
- Rectangle → Replace with letters in same row but opposite corners.

# PLAYFAIR CIPHER...

## Step 1: Create the 5×5 Matrix

- Remove duplicate letters from the key.
- Combine **I** and **J** in one cell.
- Fill remaining letters alphabetically.
- Key: MONARCHY

## Step 2: Prepare Plaintext: **BALLOONS**

- Rule: If a pair has the same letters → insert **X** between them.
- Split: BA | LX | LO | ON | SX

|          |          |          |              |          |
|----------|----------|----------|--------------|----------|
| <b>M</b> | <b>O</b> | <b>N</b> | <b>A</b>     | <b>R</b> |
| <b>C</b> | <b>H</b> | <b>Y</b> | <b>B</b>     | <b>D</b> |
| <b>E</b> | <b>F</b> | <b>G</b> | <b>I / J</b> | <b>K</b> |
| <b>L</b> | <b>P</b> | <b>Q</b> | <b>S</b>     | <b>T</b> |
| <b>U</b> | <b>V</b> | <b>W</b> | <b>X</b>     | <b>Z</b> |



# PLAYFAIR CIPHER...

## ENCRYPTION RULES

1. If both letters are in the same row, replace each letter with the letter to its right.
2. If both letters are in the same column, replace each letter with the letter below it.
3. If the letters form a rectangle, replace each letter with the letter in the same row but the other column.

|   |   |   |     |   |
|---|---|---|-----|---|
| M | O | N | A   | R |
| C | H | Y | B   | D |
| E | F | G | I/J | K |
| L | P | Q | S   | T |
| U | V | W | X   | Z |

|            |    |    |    |    |    |
|------------|----|----|----|----|----|
| Plaintext  | BA | LX | LO | ON | SX |
| Ciphertext | IB | SU | PM | NA | XA |

# PLAYFAIR CIPHER...

## Advantages

- It encrypts pairs of letters instead of single letters, making it more secure than simple substitution.
- It hides single-letter frequency patterns better.
- It was considered strong for manual encryption in the past.

## Disadvantages

- It is more complex and slower to use manually.
- It can still be broken using frequency analysis of letter pairs.
- It requires creating and memorizing a matrix.

# RAIL FENCE CIPHER

- The **Rail Fence Cipher** is a type of **transposition cipher**, which means it doesn't change the letters themselves, but changes their **order** according to a pattern. Let me explain in detail and simply.

## How it Works

- The idea is to write the message in a zigzag pattern across multiple “rails” (rows), then read it row by row to create the ciphertext.

## Steps:

- **Choose the number of rails** (rows).
- **Write the message diagonally across the rails.** Move down one rail at a time, then move back up, like a zigzag.
- **Read the letters row by row** to get the encrypted message.

# RAIL FENCE CIPHER

## Example

- **Message:** HELLO WORLD  
**Number of Rails:** 3
- Step 1: Remove spaces (optional) → HELLOWORLD
- Step 2: Write in zigzag across 3 rails:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| H |   |   |   | O |   |   |   | L |   |
|   | E |   | L |   | W |   | R |   | D |
|   |   | L |   |   |   | O |   |   |   |

Cipher text- HOLELWRDLO

# RAIL FENCE CIPHER

## Advantages

- It is simple and fast to use.
- It does not change the letters themselves (only rearranges positions).
- It is easy to implement without special tools.

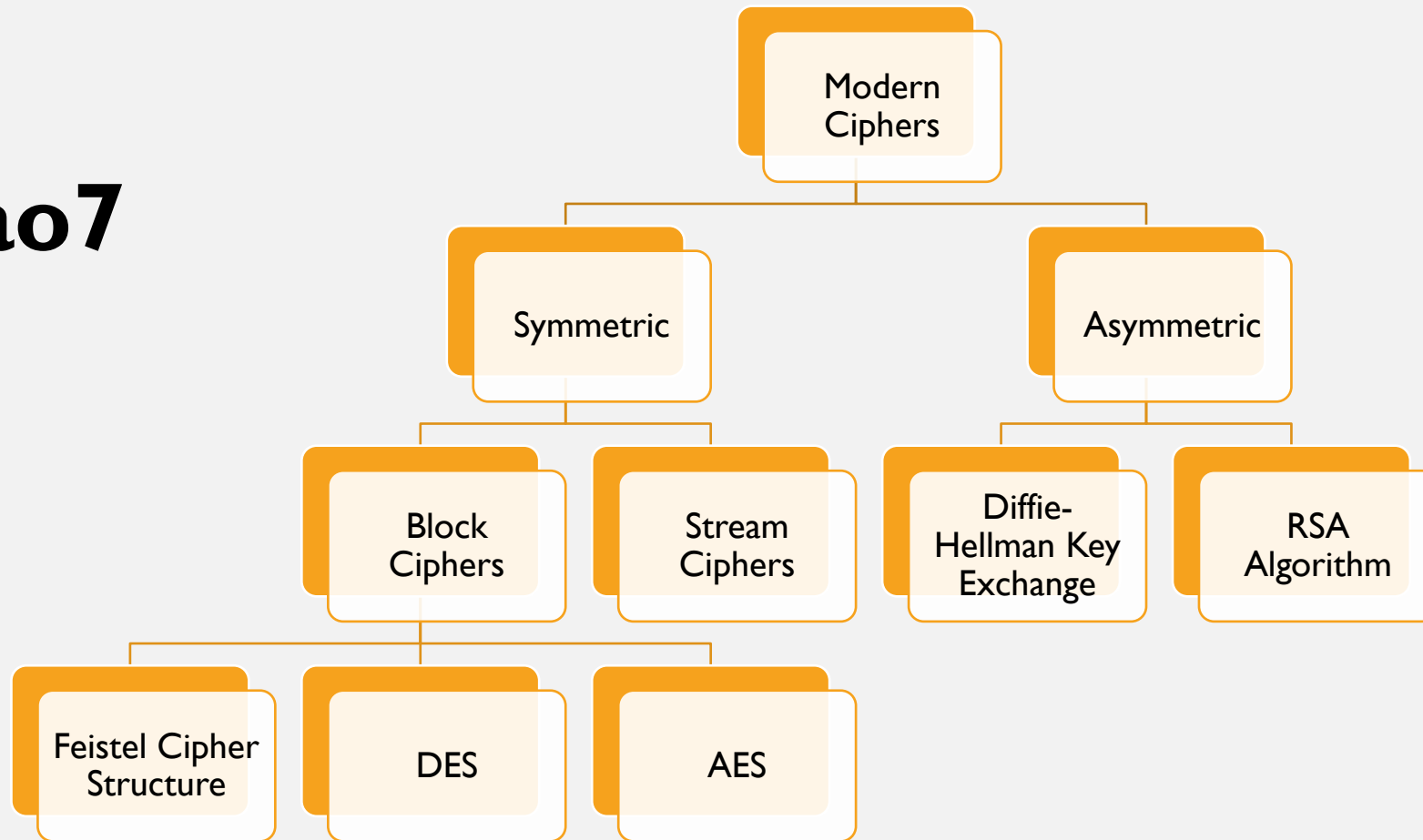
## Disadvantages

- It provides very weak security.
- It can be broken easily by trying different rail numbers.
- It does not hide letter frequency at all.



# MODERN CIPHERS

66wzzao7



# SYMMETRIC ENCRYPTION

## Introduction to Symmetric Encryption

- **Definition:** Symmetric encryption, also called **conventional** or **single-key encryption**, is a method where the same key is used for both encryption and decryption.
- **History:** It was the **only type of encryption** before public-key encryption appeared in the 1970s.
- **Current Use:** Despite public-key methods, symmetric encryption is still **widely used** because it is fast and efficient.



# SYMMETRIC CIPHER MODEL

A symmetric encryption system has **five essential components**:

- a. Plaintext (X):** Input message/data.
- b. Encryption Algorithm (E):** A procedure that converts plaintext into ciphertext using the secret key.
- c. Secret Key (K):** A unique value used by both sender and receiver. Determines **how the algorithm transforms plaintext**.
- d. Ciphertext (Y):** The scrambled output message. Appears random and unreadable without the key.
- e. Decryption Algorithm (D):** Essentially the encryption algorithm run in reverse, restores plaintext using the same key.

# SYMMETRIC CIPHER MODEL...

## Mathematical Representation

- **Encryption:**

$$Y = E(K, X)$$

- **Decryption:**

$$X = D(K, Y)$$

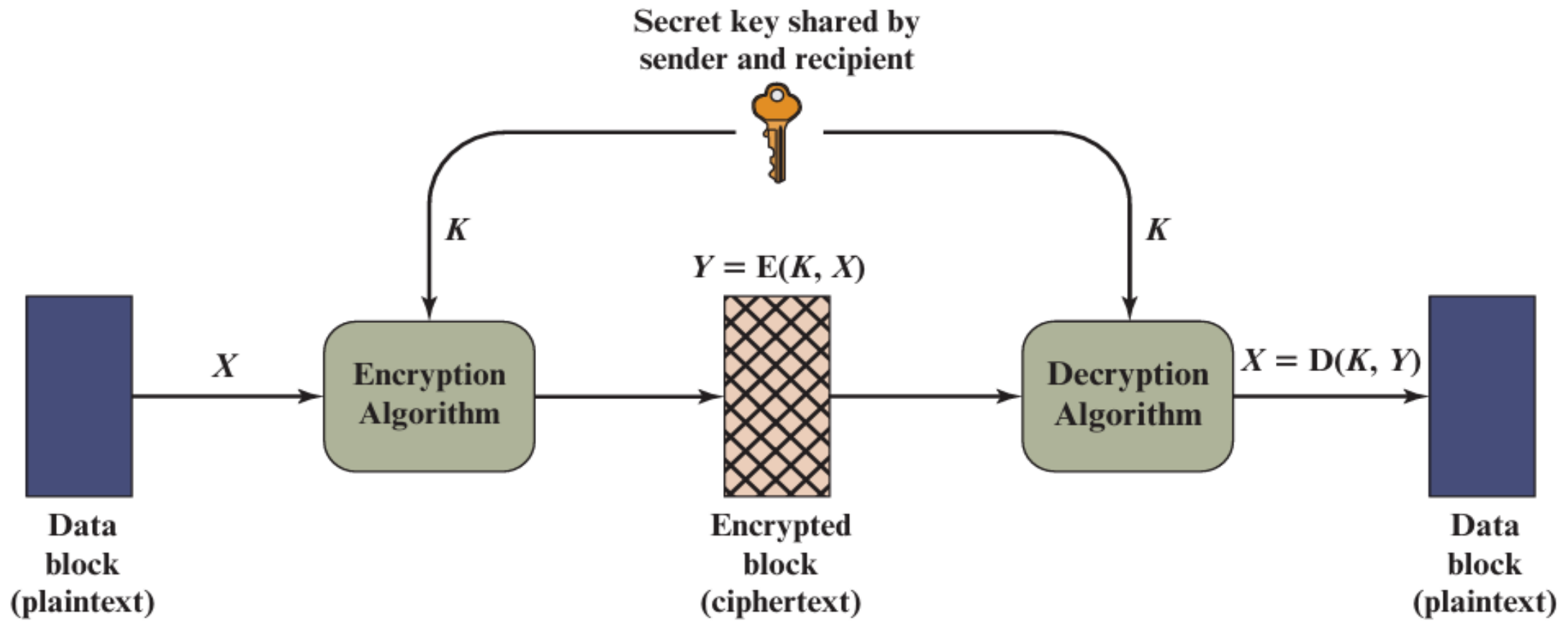
- **Symmetric cipher** simply means:

**‘An encryption method where the same secret key is used to both encrypt and decrypt the message.’**

- In even simpler words:

**One key locks the message, and the same key unlocks it.**

# SYMMETRIC CIPHER MODEL...



**Figure 3.1** Simplified Model of Symmetric Encryption

# ASYMMETRIC CIPHER MODEL

- Uses two different keys for encryption and decryption
- Also called **public-key cryptography**
- Developed to solve the key distribution problem of symmetric encryption

## **Asymmetric Cipher:**

An encryption method where:

- **Public Key** → used to encrypt the message
- **Private Key** → used to decrypt the message

In simple, ***One key locks, another key unlocks.***

# ASYMMETRIC CIPHER MODEL...

## Basic Components

- Plaintext (Original message)
- Encryption Algorithm
- Public Key (PU)
- Private Key (PR)
- Ciphertext (Encrypted message)
- Decryption Algorithm

## Mathematical Model

Encryption:

$$Y = E(PU, X)$$

Decryption:

$$X = D(PR, Y)$$

Where:

- $X$  = Plaintext
- $Y$  = Ciphertext
- $PU$  = Public Key
- $PR$  = Private Key

# ASYMMETRIC CIPHER MODEL...

## ■ Working Process

**Step 1:** Receiver generates public and private keys

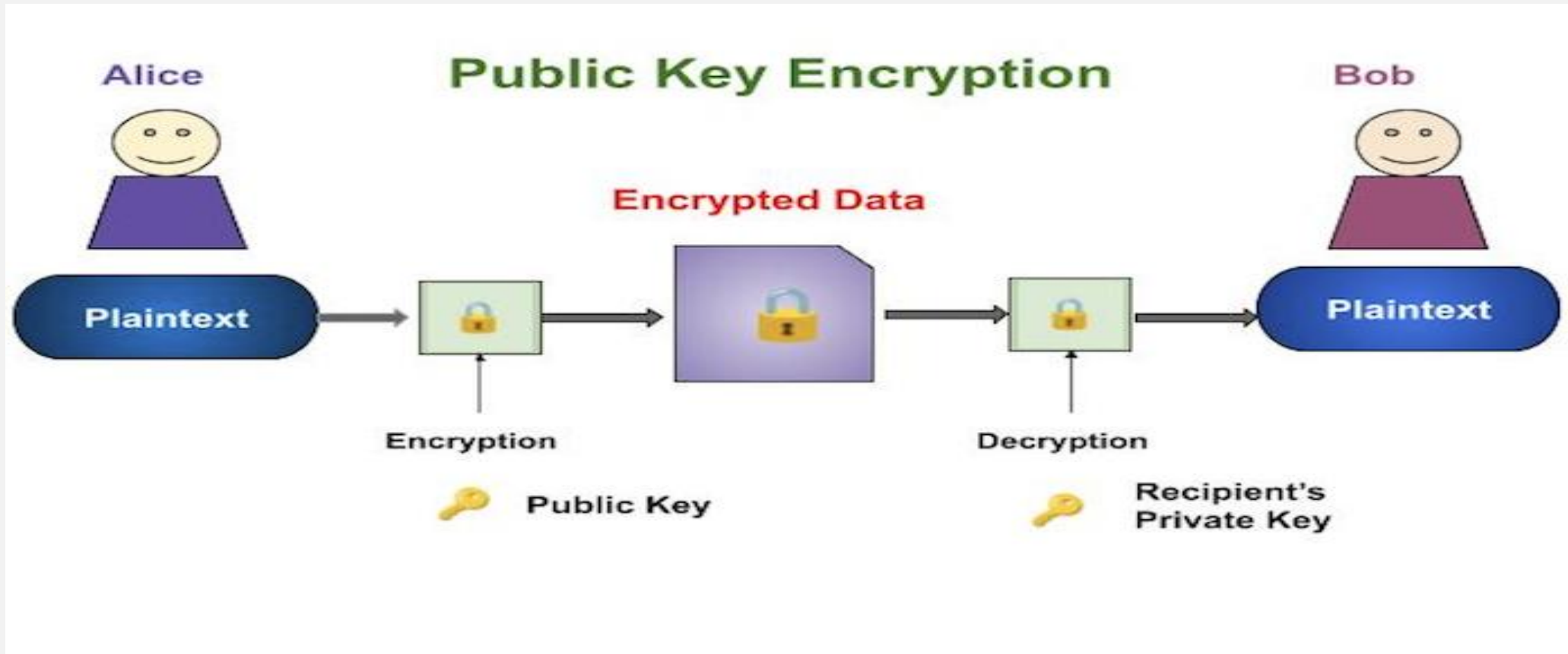
**Step 2:** Public key is shared with sender

**Step 3:** Sender encrypts message using public key

**Step 4:** Ciphertext is transmitted

**Step 5:** Receiver decrypts using private key

# ASYMMETRIC CIPHER MODEL...



# DIFFERENCE

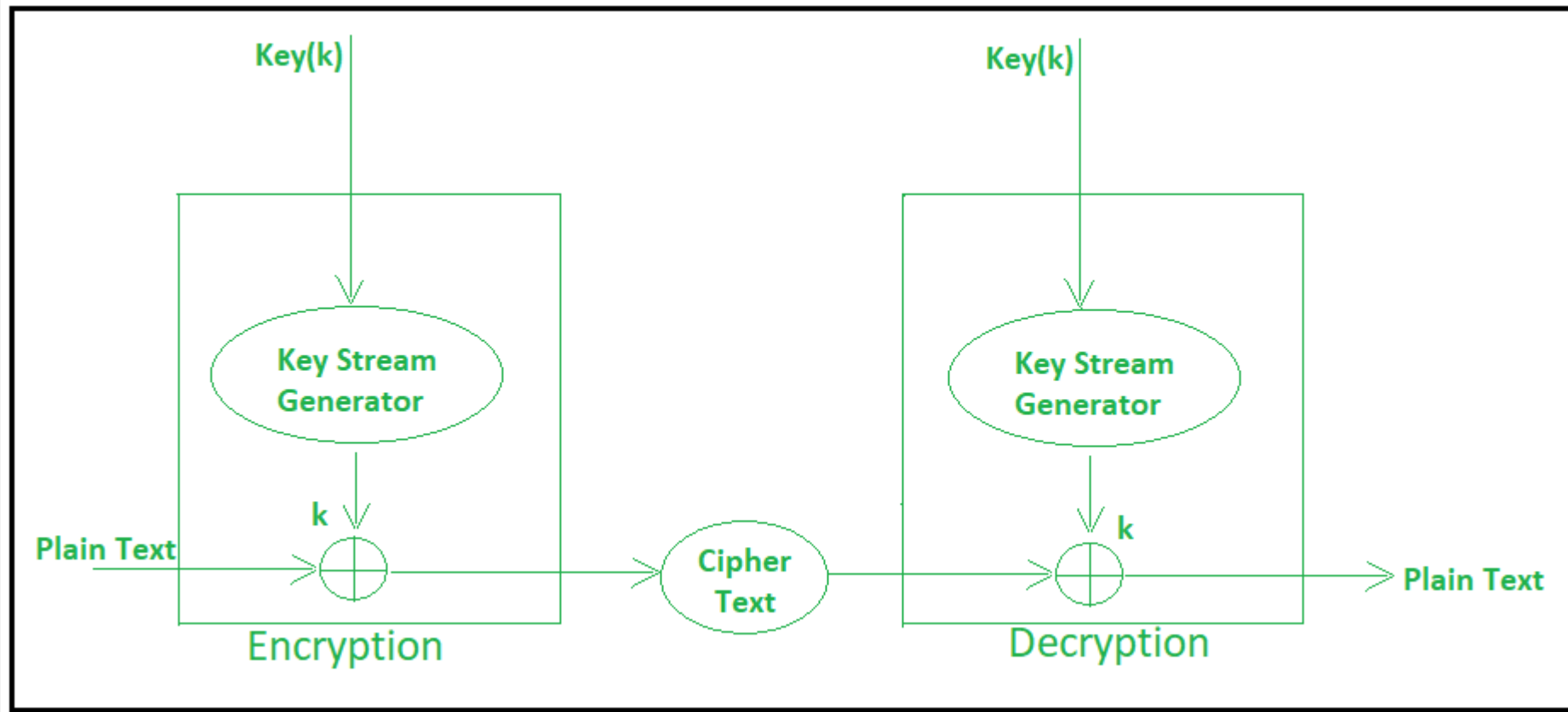
| Feature                  | Symmetric Encryption                                   | Asymmetric Encryption                               |
|--------------------------|--------------------------------------------------------|-----------------------------------------------------|
| Definition               | Uses <b>one same key</b> for encryption and decryption | Uses <b>two different keys</b> (public and private) |
| Number of Keys           | Single key                                             | Two keys                                            |
| Key Sharing              | Key must be shared securely                            | Public key can be shared openly                     |
| Speed                    | Faster                                                 | Slower                                              |
| Security Dependency      | Depends on secrecy of shared key                       | Depends on secrecy of private key                   |
| Computation              | Less computational power required                      | More computational power required                   |
| Key Distribution Problem | Difficult                                              | Easier                                              |
| Usage                    | Encrypting large data                                  | Key exchange, authentication, digital signatures    |
| Examples                 | AES, DES,                                              | RSA, Diffie-Hellman, ECC                            |
| Common Use               | File encryption, VPN, storage                          | HTTPS, email security, digital signatures           |



# STREAM CIPHERS

- A **stream cipher** is a type of symmetric encryption algorithm in which data is encrypted **one bit or one byte at a time**. This is different from block ciphers, which encrypt data in **fixed-size blocks (commonly 128 bits)**.
- In a stream cipher, the encryption process begins with a **secret key (K)** that is given as input to a **pseudorandom bit generator (PRBG)**.
- This generator produces a sequence of random-looking bits called a **keystream**. The keystream is then combined with the plaintext using an **XOR (exclusive OR) operation** to produce the ciphertext.
- Because stream ciphers process data **bit-by-bit or byte-by-byte**, they are generally **very fast** and efficient.
- For this reason, they are commonly used in **real-time communication systems**, such as video streaming, voice calls, and online gaming, where data needs to be encrypted and transmitted immediately.

# STREAM CIPHERS...



# STREAM CIPHERS...

## Encryption Process

- During encryption, the **plaintext** is combined with the **keystream** using the **XOR operation**.
- The XOR operation compares two bits and produces the following results:

## Example

- Plaintext:    1 0 0 1 1 0 0 1
- Keystream:   1 1 0 0 0 0 1 1
- Ciphertext:  0 1 0 1 1 0 1 0

| Bit A | Bit B | A XOR B |
|-------|-------|---------|
| 0     | 0     | 0       |
| 0     | 1     | 1       |
| 1     | 0     | 1       |
| 1     | 1     | 0       |

# STREAM CIPHERS...

## Advantages of Stream Cipher

- **High Speed** – Stream ciphers encrypt data **bit-by-bit or byte-by-byte**, making them generally faster than block ciphers.
- **Low Complexity** – They are **simple to design and implement**, often requiring fewer lines of code and less computational power.
- **Efficient for Real-Time Applications** – Stream ciphers can encrypt data **as it is generated**, which makes them suitable for real-time communication such as video streaming, voice communication, and online gaming.
- **Low Hardware Requirements** – They do not require complex hardware, so they work well in **devices with limited processing power**.
- **Continuous Data Encryption** – They allow **continuous encryption of data streams**, which is useful when the total size of the data is not known in advance.

# BLOCK CIPHERS

- A **block cipher** is a type of symmetric encryption algorithm that encrypts data in **fixed-size blocks instead of individual bits or bytes**.
- In most modern systems, the plaintext is divided into blocks, and each block is encrypted separately using a **secret key** to produce ciphertext.
- Block ciphers encrypt data in big fixed size chunks called blocks.
- Sometimes the last bit of data is too small. It doesn't fill a whole block. So, we add extra bits. This addition of extra bits are called padding. This makes the block the right size. Then we encrypt it.
- Block ciphers use a series of **mathematical operations and substitution-permutation techniques** to transform plaintext into ciphertext.
- The same key is used for both **encryption and decryption**, which makes block ciphers part of **symmetric-key cryptography**.

# BLOCK CIPHERS...

- **How Block Cipher Works**
- The plaintext message is divided into **fixed-size blocks** (for example, 128 bits).
- Each block is processed using the **encryption algorithm and secret key**.
- The algorithm performs multiple rounds of **substitution, permutation, and mixing operations**.
- The output of each block becomes the **ciphertext block**.

# DIFFERENCE BETWEEN BLOCK CIPHER AND STREAM CIPHER

| Basis             | Block Cipher                                                                               | Stream Cipher                                                                            |
|-------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Definition        | A block cipher encrypts data in <b>fixed-size blocks</b> (e.g., 64 or 128 bits) at a time. | A stream cipher encrypts data <b>one bit or one byte at a time</b> .                     |
| Encryption Method | The plaintext is divided into blocks and each block is encrypted using the same key.       | Plaintext bits are encrypted continuously using a <b>keystream</b> generated from a key. |
| Speed             | Generally <b>slower</b> because it processes larger blocks.                                | Usually <b>faster</b> because it encrypts data bit-by-bit or byte-by-byte.               |
| Complexity        | More complex due to block processing and different modes (ECB, CBC, etc.).                 | Simpler structure with continuous encryption.                                            |
| Error Propagation | An error in one block may affect the entire block.                                         | An error usually affects only <b>one bit or byte</b> of data.                            |
| Examples          | <b>AES, DES, Blowfish</b>                                                                  | <b>RC4, Salsa20, ChaCha20</b>                                                            |
| Usage             | Commonly used for <b>file encryption and database security</b> .                           | Often used for <b>real-time communication like streaming and wireless encryption</b> .   |

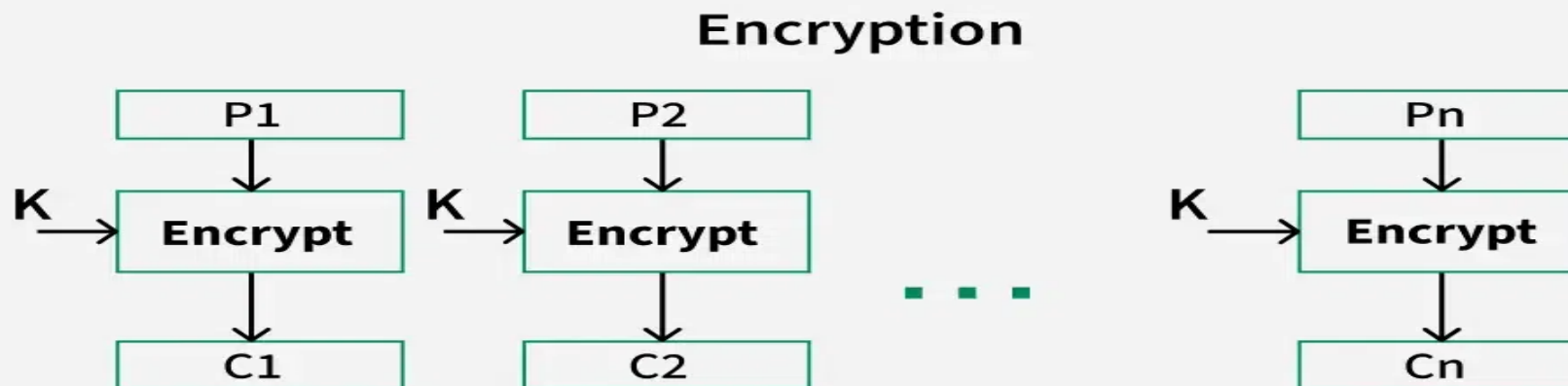
# BLOCK CIPHER MODES OF OPERATION

- Block Cipher Modes of Operation define how to securely encrypt and decrypt large amounts of data using a block cipher.
- A block cipher is an encryption algorithm that processes data in fixed-size blocks (e.g., 128 bits) rather than one bit at a time.
- However, to encrypt data larger than a single block, different modes of operation are used to ensure both security and efficiency.
- Here are a few common modes:
  1. **Electronic Code Book (ECB)**
  2. **Cipher Block Chaining**
  3. **Cipher Feedback Mode (CFB)**
  4. **Output Feedback Mode (OFB)**



# ELECTRONIC CODE BOOK (ECB)

- **Electronic Code Book (ECB)** is the simplest mode of operation for a block cipher. In this mode, the plaintext message is divided into fixed-size blocks (for example, 64 or 128 bits).
- Each block is encrypted separately using the same key to produce the corresponding ciphertext block.
- If the message is longer than the block size, it is broken into multiple blocks and each block is encrypted independently using the same encryption algorithm.



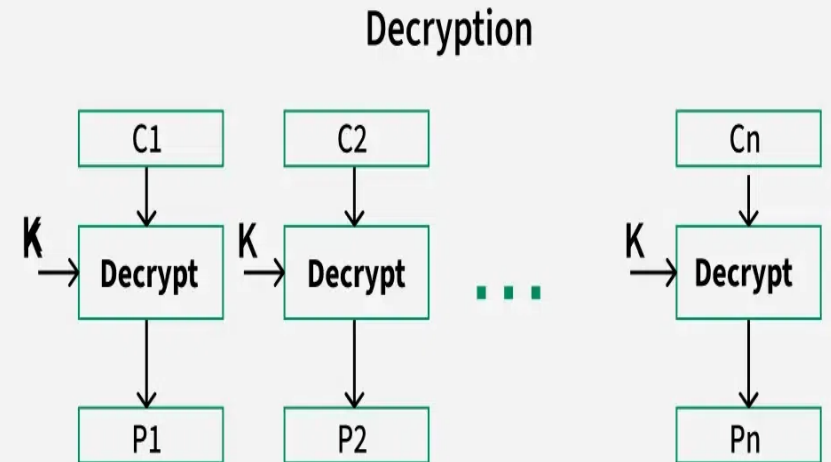
# ELECTRONIC CODE BOOK (ECB)

## Advantages of ECB

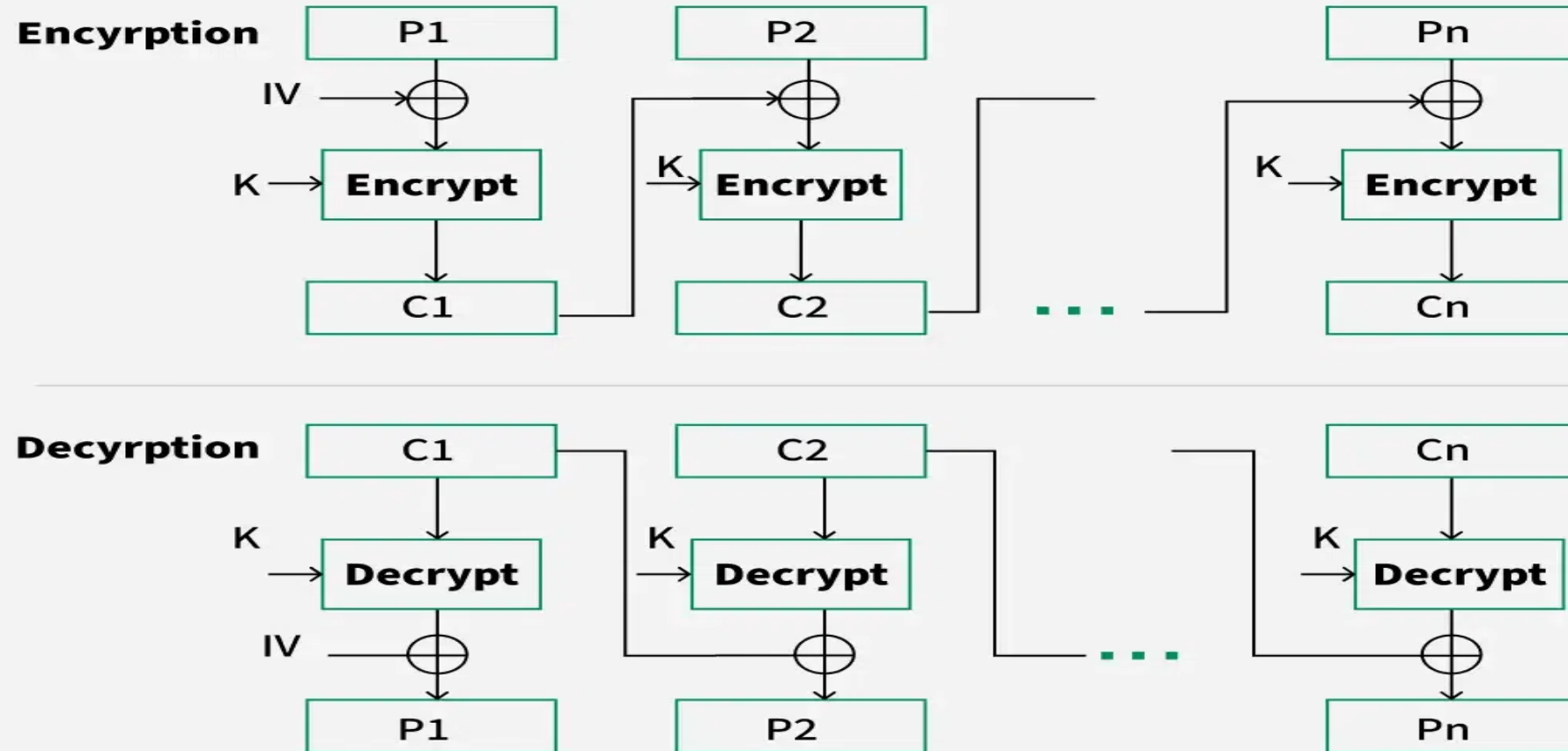
- **Parallel encryption is possible**, because each plaintext block is encrypted independently of other blocks.
- **Faster encryption**, since multiple blocks can be processed at the same time.
- **Simple to implement**, as it directly encrypts each plaintext block without additional operations.

## Disadvantages of ECB

- **Vulnerable to cryptanalysis**, because there is a direct relationship between plaintext and ciphertext.
- **Identical plaintext blocks produce identical ciphertext blocks**, which can reveal patterns in the encrypted message.
- **Not suitable for highly secure systems**, since attackers may analyze repeated patterns.



# CIPHER BLOCK CHAINING (CBC)



# CIPHER BLOCK CHAINING (CBC)

- **Cipher Block Chaining (CBC)** is an improved block cipher mode designed to overcome the security weaknesses of ECB.
- In CBC mode, each plaintext block is first combined with the previous ciphertext block using the **XOR operation** before being encrypted.
- The first block uses an **Initialization Vector (IV)** instead of a previous ciphertext block. This chaining process ensures that even if two plaintext blocks are identical, their ciphertext blocks will be different.

# CIPHER BLOCK CHAINING (CBC)

## Advantages of CBC

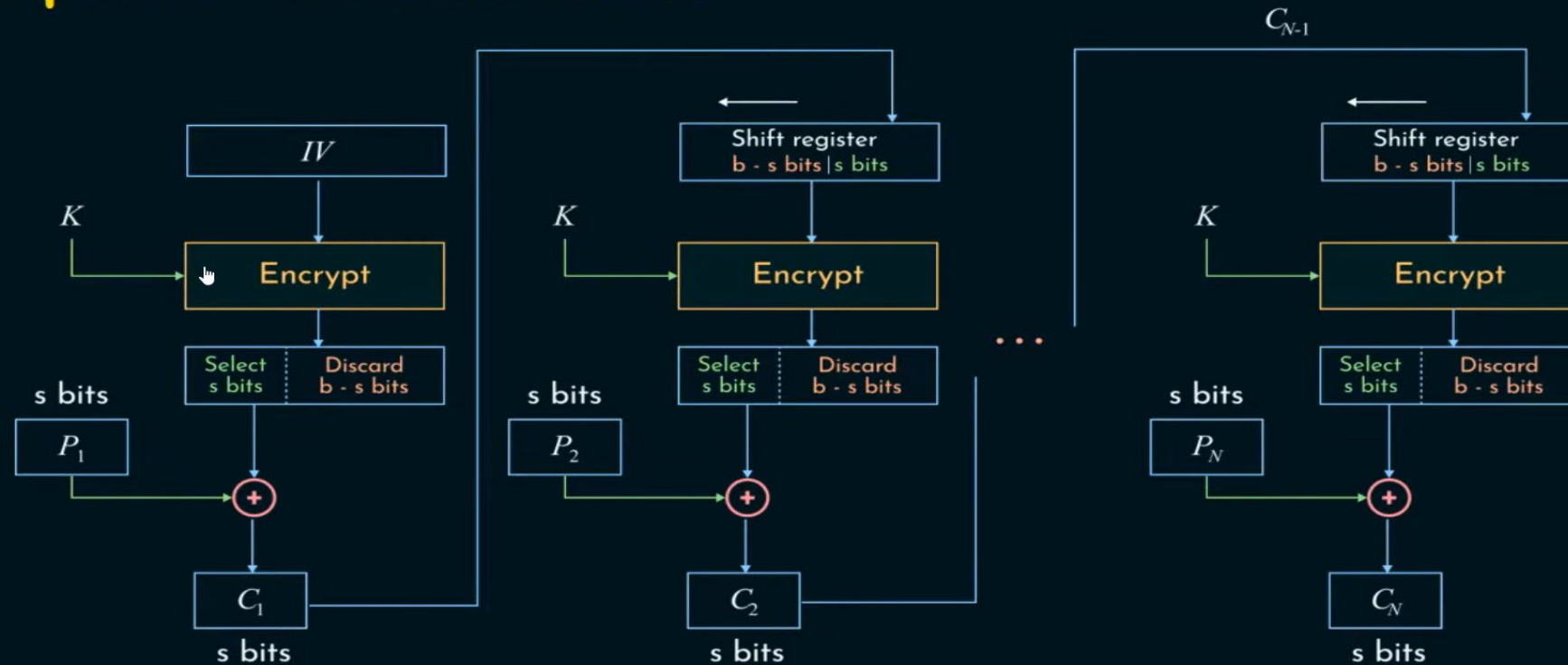
- **More secure than ECB**, because it hides patterns in the plaintext.
- **Works well for messages larger than the block size (b bits).**
- **Provides better resistance to cryptanalysis** compared to ECB.
- **Can be used as an authentication mechanism** in some systems.

## Disadvantages of CBC

- **Encryption and decryption depend on the previous ciphertext block**, which means blocks cannot be processed completely in parallel.
- **Error propagation can occur**, where an error in one block may affect the decryption of the next block.

# CIPHER FEEDBACK MODE (CFB)

## Cipher Feedback Mode



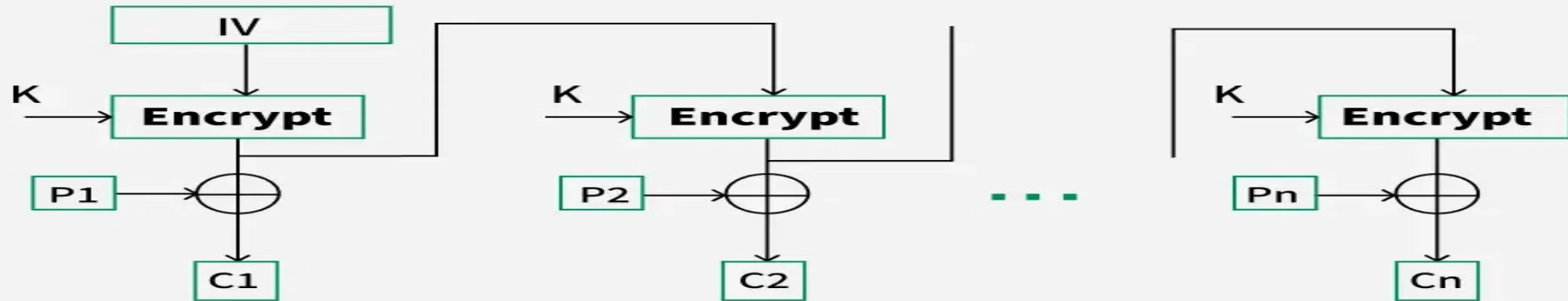
CFB - Encryption

## CIPHER FEEDBACK MODE (CFB)

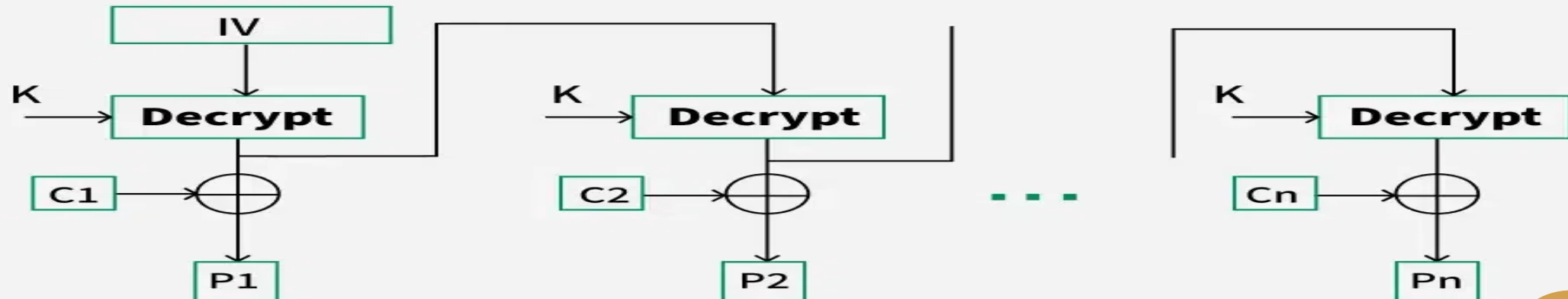
- In this mode the cipher is given as feedback to the next block of encryption with some new specifications: first, an initial vector IV is used for first encryption and output bits are divided as a set of  $s$  and  $b-s$  bits.
- The left-hand side  $s$  bits are selected along with plaintext bits to which an XOR operation is applied.
- The result is given as input to a shift register having  $b-s$  bits to lhs,  $s$  bits to rhs and the process continues.
- The encryption and decryption process for the same is shown below, both of them use encryption algorithms.

# OUTPUT FEEDBACK MODE

## Encryption



## Decryption

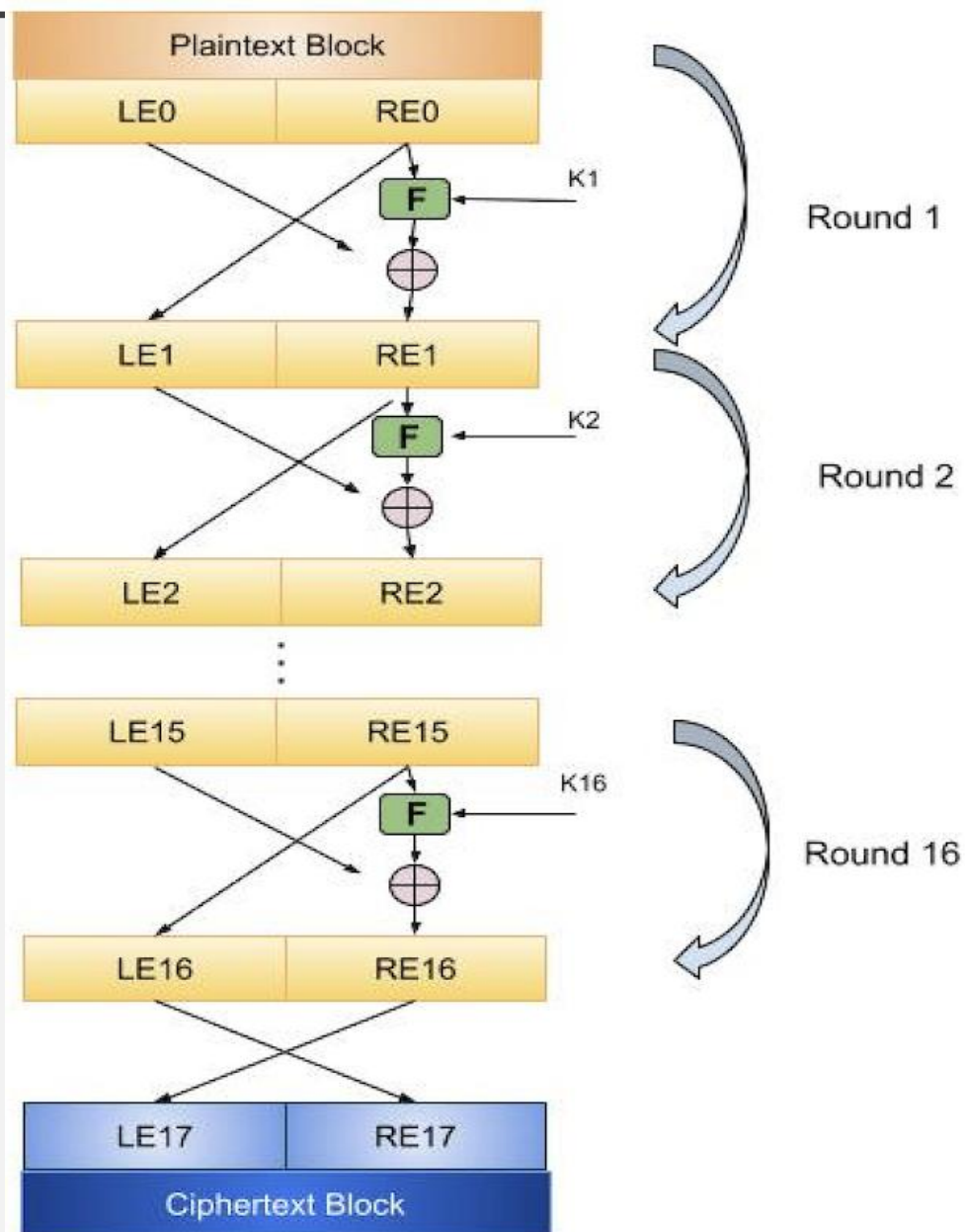




## OUTPUT FEEDBACK MODE

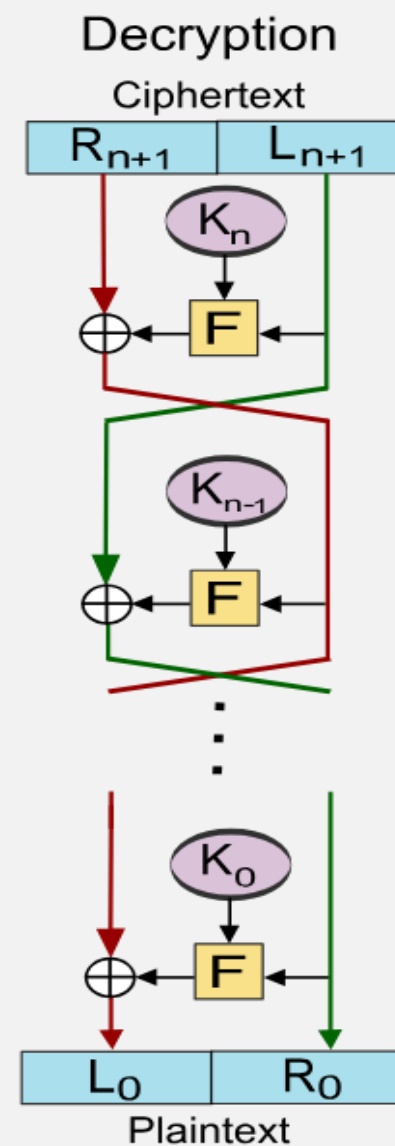
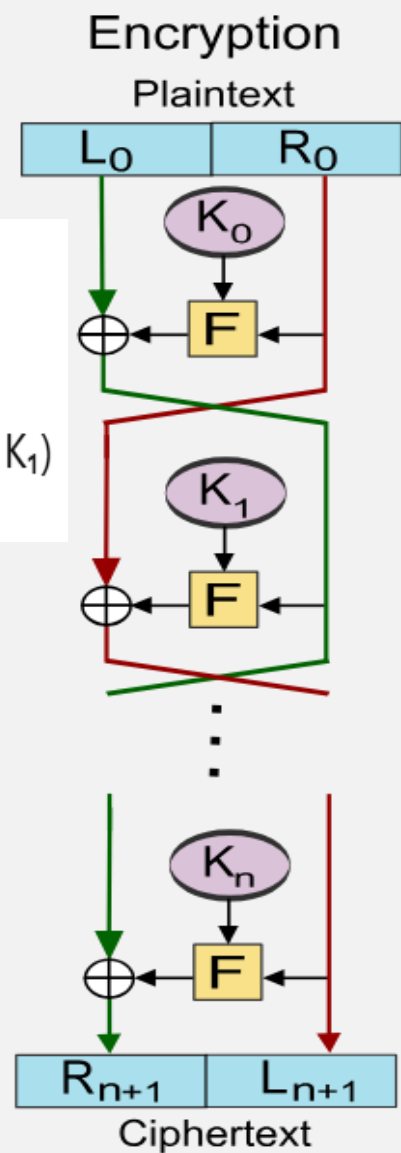
- The output feedback mode follows nearly the same process as the Cipher Feedback mode except that it sends the encrypted output as feedback instead of the actual cipher which is XOR output.
- In this output feedback mode, all bits of the block are sent instead of sending selected  $s$  bits.
- The Output Feedback mode of block cipher holds great resistance towards bit transmission errors.
- It also decreases the dependency or relationship of the cipher on the plaintext.

# FEISTEL CIPHER



## ◆ Round 1:

- $L_1 = R_0$
- $R_1 = L_0 \oplus F(R_0, K_1)$



# FEISTEL CIPHER

- The input block in each round is divided into two parts called L (left) and R (right).
- In every round, the right half R remains unchanged at first, while the left half L is processed using R and the key.
- A function  $f$  is applied to R and the key  $K$  to produce an output  $f(R, K)$ . This output is then XORed with L to generate a new value.
- After this, a swapping (permutation) step takes place. The left and right halves exchange their positions. So, the new left half becomes the previous right half, and the new right half becomes the result obtained from modifying L.
- These substitution and permutation operations together form one round, and the total number of rounds depends on the design of the algorithm.
- After completing the final round, the two halves are combined in the order R followed by L to produce the final ciphertext block.

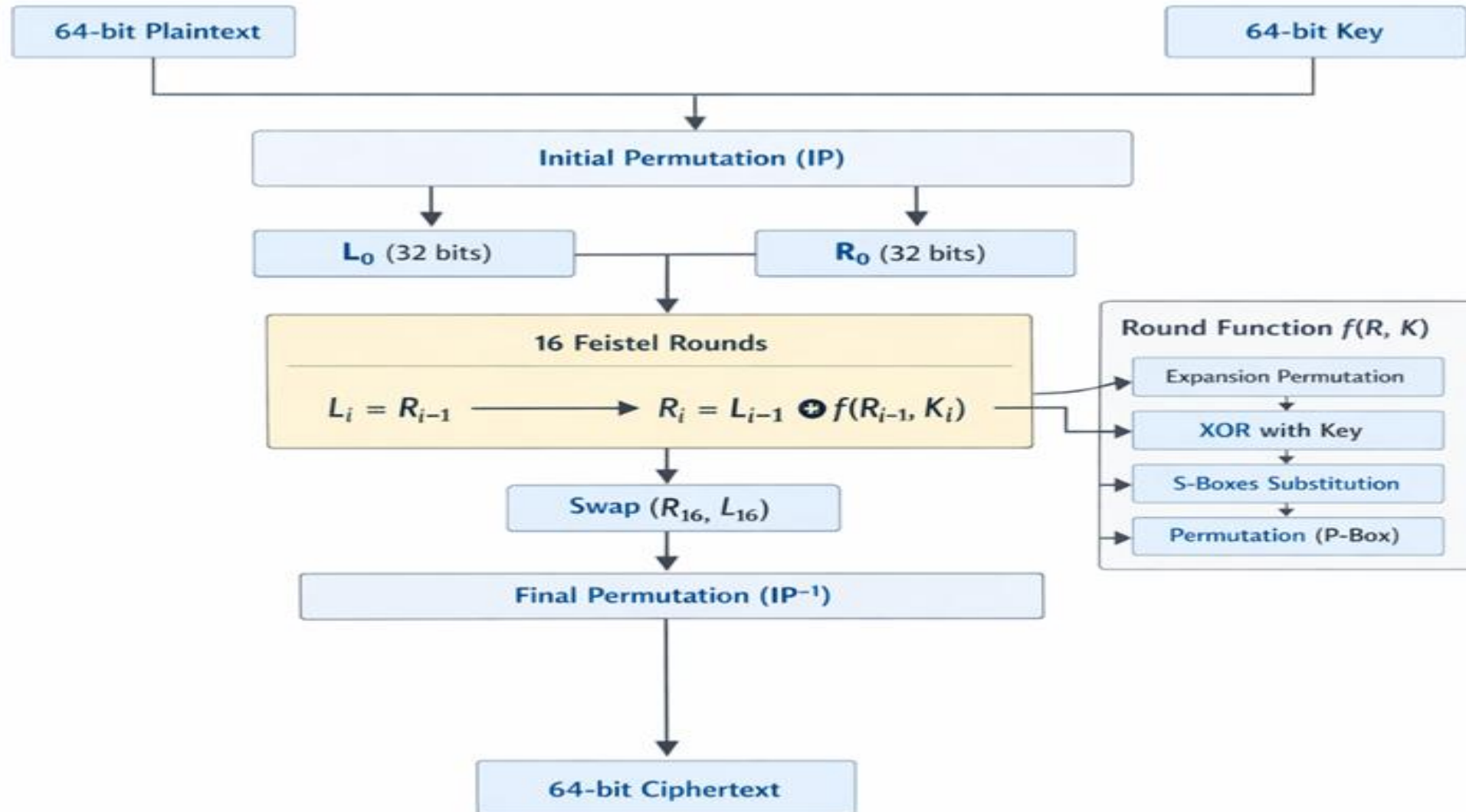
## WHY IS FEISTEL CIPHER IMPORTANT?

- It provides a flexible framework to build strong ciphers.
- Many real-world ciphers (like DES) are based on this model.
- Ensures reversibility (you can always get back plaintext from ciphertext).
- Makes encryption/decryption more efficient and practical.

# DATA ENCRYPTION STANDARD (DES)

- The Data Encryption Standard (DES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST).
- DES is an implementation of a Feistel Cipher. It uses 16 round Feistel structure.
- The block size is 64-bit. Though, key length is 64-bit, DES has an effective key length of 56 bits, since 8 of the 64 bits of the key are not used by the encryption algorithm (function as check bits only).

# DATA ENCRYPTION STANDARD (DES)



# DATA ENCRYPTION STANDARD....

- In the first step, the 64-bit plain text block is handed over to an initial Permutation (IP) function.
- The initial permutation performed on plain text.
- Next the initial permutation (IP) produces two halves of the permuted block; says Left Plain Text (LPT) and Right Plain Text (RPT).
- Now each LPT and RPT to go through 16 rounds of encryption process.
- In the end, LPT and RPT are rejoined and a Final Permutation (FP) is performed on the combined block.
- The result of this process produces 64-bit cipher text.

# DATA ENCRYPTION STANDARD....

## Step -1- Initial Permutation (IP)

- As we have noted, the Initial permutation (IP) happens only once and it happens before the first round. It suggests how the transposition in IP should proceed, as show in figure.
- For example, it says that the IP replaces the first bit of the original plain text block with the 58th bit of the original plain text, the second bit with the 50th bit of the original plain text block and so on.
- This is nothing but jugglery of bit positions of the original plain text block. The same rule applies for all the other bit positions which shows in the figure.



# DATA ENCRYPTION STANDARD....

## IP Table

|    |    |    |    |    |    |    |   |
|----|----|----|----|----|----|----|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57 | 49 | 41 | 33 | 25 | 17 | 9  | 1 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 7 |

| Input Position and bits |             |    |    |    |    |    |    |
|-------------------------|-------------|----|----|----|----|----|----|
| 1 2- H bit              | 3           | 4  | 5  | 6  | 7  | 8  |    |
| 9 10-G bit              | 11          | 12 | 13 | 14 | 15 | 16 |    |
| 17 18-F bit             | 19          | 20 | 21 | 22 | 23 | 24 |    |
| 25                      | 26          | 27 | 28 | 29 | 30 | 31 | 32 |
| 33 34-D bit             | 35 36-E bit |    | 37 | 38 | 39 | 40 |    |
| 41 42-C bit             | 43          | 44 | 45 | 46 | 47 | 48 |    |
| 49 50-B bit             | 51 52-Y bit |    | 53 | 54 | 55 | 56 |    |
| 57 58-A bit             | 59 60-X bit |    | 61 | 62 | 63 | 64 |    |

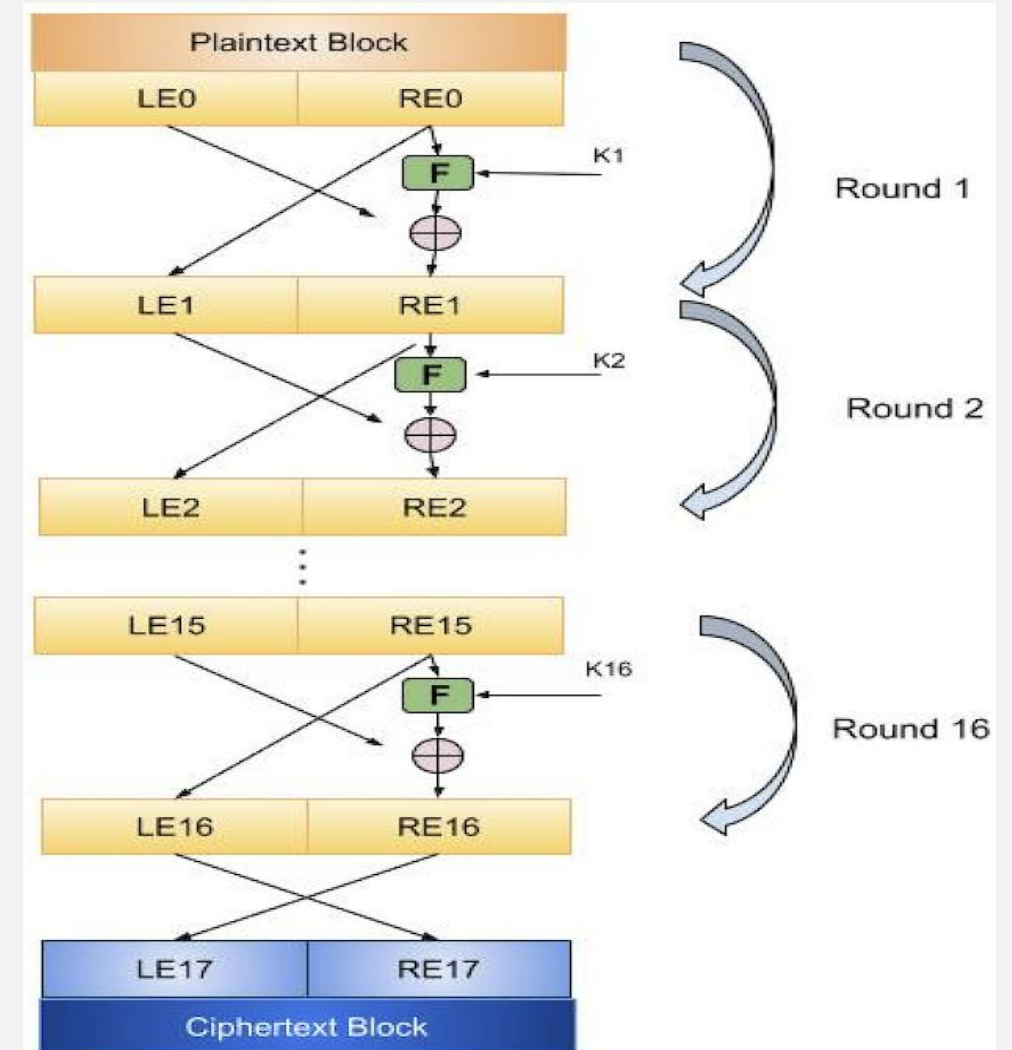
| Output B |         |         |         |         |         |         |         |
|----------|---------|---------|---------|---------|---------|---------|---------|
| Column1  | Column2 | Column3 | Column4 | Column5 | Column6 | Column7 | Column8 |
| A        | B       | C       | D       | E       | F       | G       | H       |
| X        | Y       |         |         |         |         |         |         |
|          |         |         |         |         |         |         |         |
|          |         |         |         |         |         |         |         |
|          |         |         |         |         |         |         |         |

# DATA ENCRYPTION STANDARD....

## Step-2 Split 64-bit data

After the initial permutation performed on plain text:

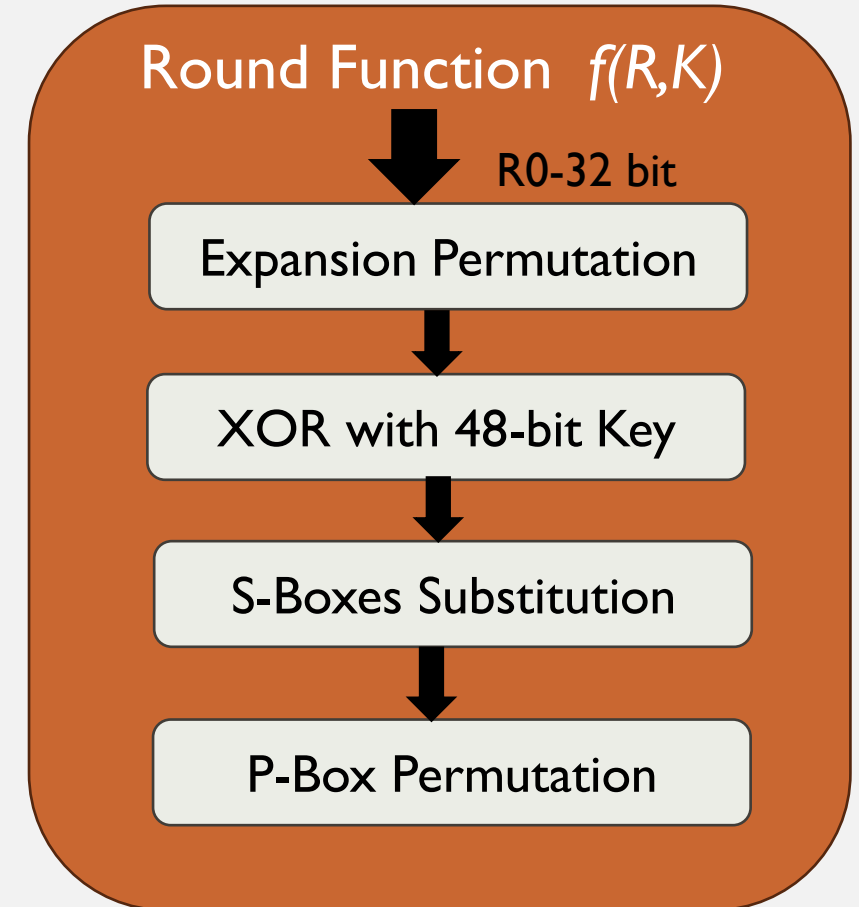
- The initial permutation (IP) produces two halves of the permuted block; says Left Plain Text (LPT) and Right Plain Text (RPT).
- Now each LPT and RPT to go through 16 rounds of encryption process similar to Feistel Cipher.



# DES...

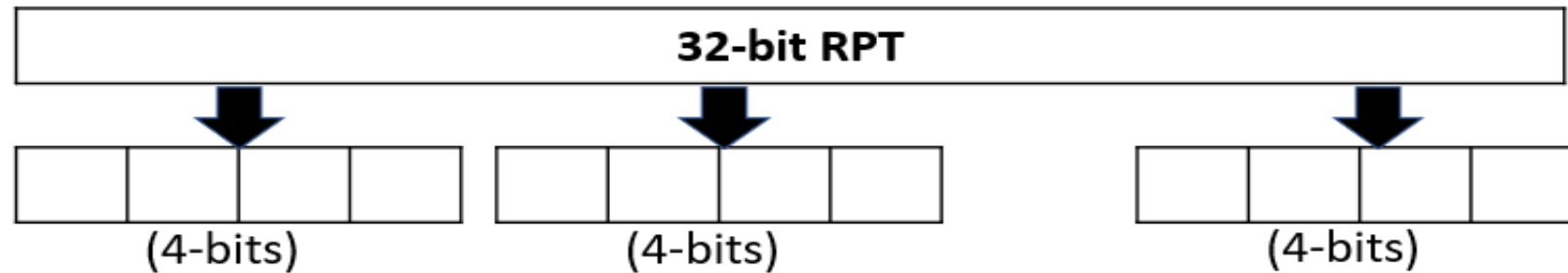
## Step-3 Round Function

- The right half (32 bits) is **expanded to 48 bits** using expansion permutation.
- The expanded bits are **XORed with the round key (48 bits)**.
- The result is passed through **S-boxes**, reducing it back to 32 bits.
- Finally, a **permutation (P-box)** rearranges the bits to produce the output of the round function.

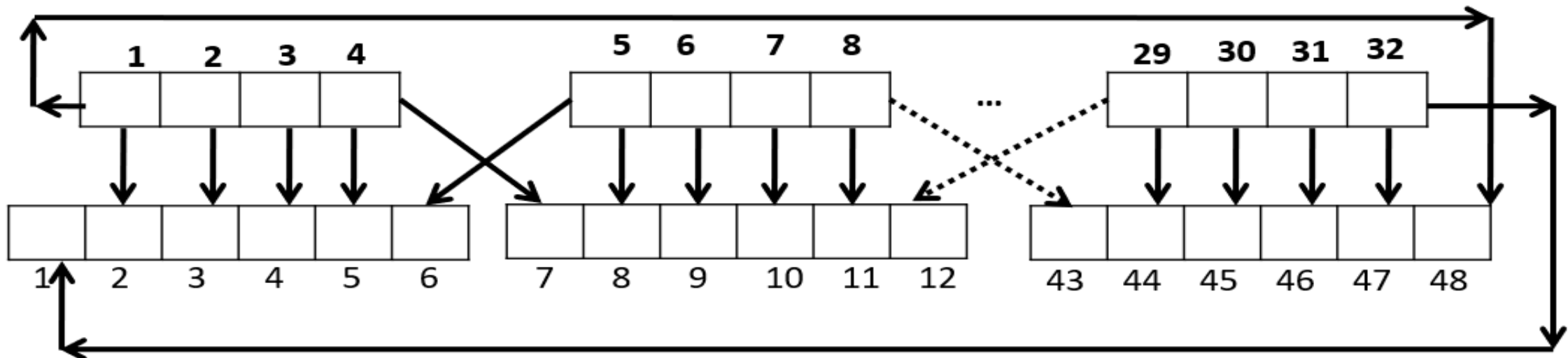


# DES....

## Step-3.1 Expansion Permutation



- Each 4-bit block is expanded to 6-bit and produce 48-bit output



# DES...

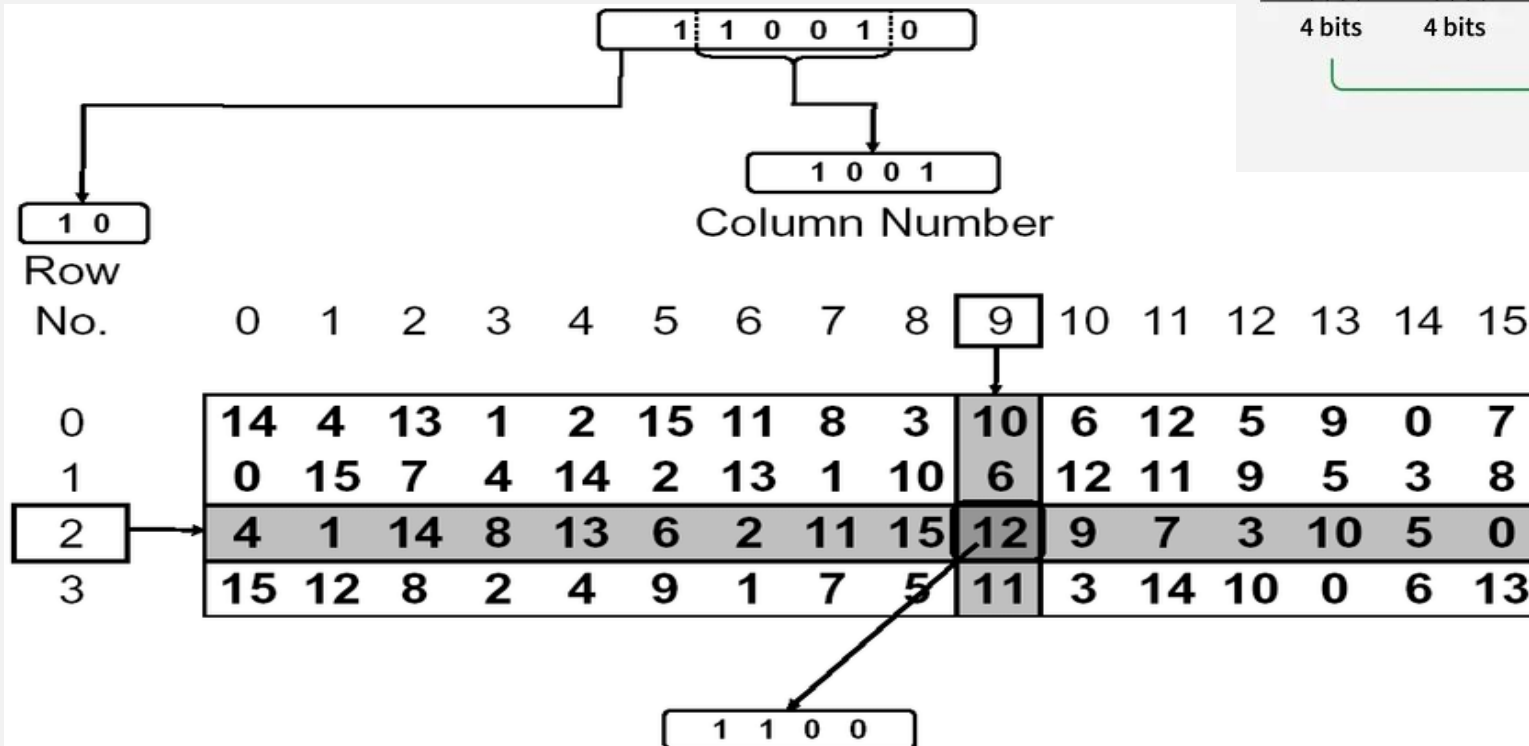
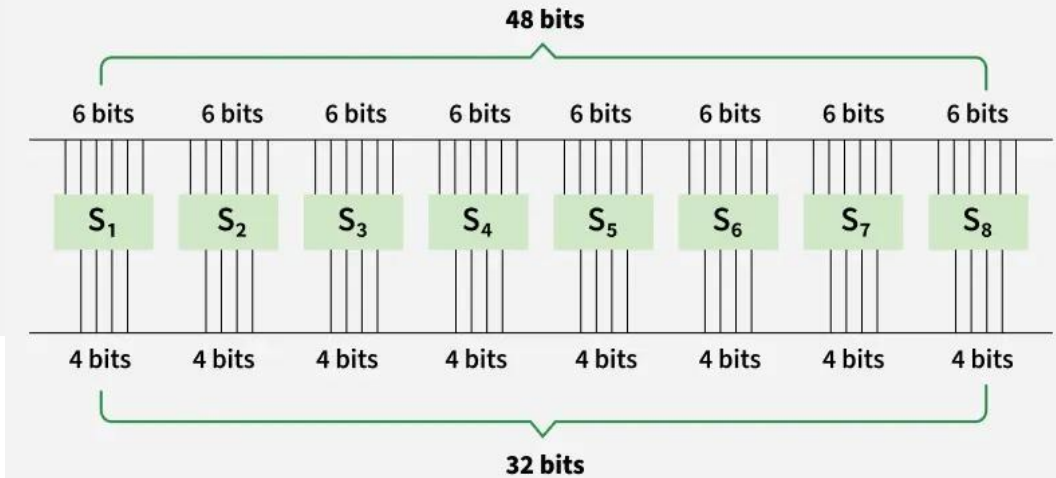
## Step 3.2 XOR with Key

- In each DES round, the expanded right half (48 bits) is **XORed with the round key  $K(n)$** .
- This step mixes the key with the data, making the encryption dependent on the key.
- It is crucial for introducing **confusion**, ensuring that the output changes significantly even with small key differences.

# DES...

## Step 3.3 S-box Compression...

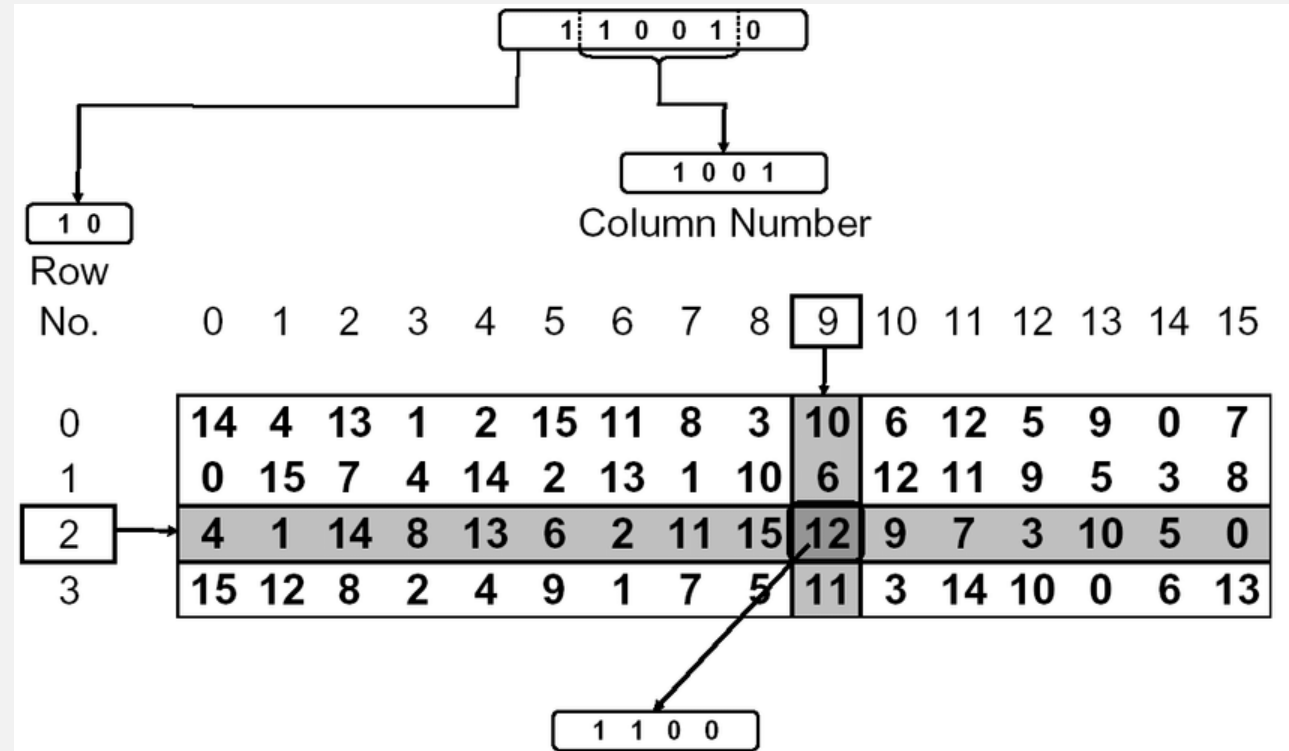
- After XOR the 48 bits is divided into **8 groups of 6 bits each**(8 groups\*6 bits=48 )



# DES...

## Inside each S-box (main logic)

- For each **6-bit input**:
  - **Step A: Find Row**
    - Take **1st and last bit**
    - Combine → gives row (0–3)
  - **Step B: Find Column**
    - Take **middle 4 bits**
    - Gives column (0–15)
- **Lookup in S-box table**
  - Each S-box is a **4 × 16 table**
  - Use (row, column) to find value



# DES...

## Step-3.4 P-box Permutation

- It is applied **after the S-box compression**.
  - S-box output = **32 bits**
- P-box takes these 32 bits and **permutes their positions**
- It does NOT change the bits (0s and 1s)
- It only changes their **order/position**(**Similar to** -Initial Permutation-We just refer to P-box table)

| P-box DES |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| <b>16</b> | <b>7</b>  | <b>20</b> | <b>21</b> | <b>29</b> | <b>12</b> | <b>28</b> | <b>17</b> |
| <b>1</b>  | <b>15</b> | <b>23</b> | <b>26</b> | <b>5</b>  | <b>18</b> | <b>31</b> | <b>10</b> |
| <b>2</b>  | <b>8</b>  | <b>24</b> | <b>14</b> | <b>32</b> | <b>27</b> | <b>3</b>  | <b>9</b>  |
| <b>19</b> | <b>13</b> | <b>30</b> | <b>6</b>  | <b>22</b> | <b>11</b> | <b>4</b>  | <b>25</b> |

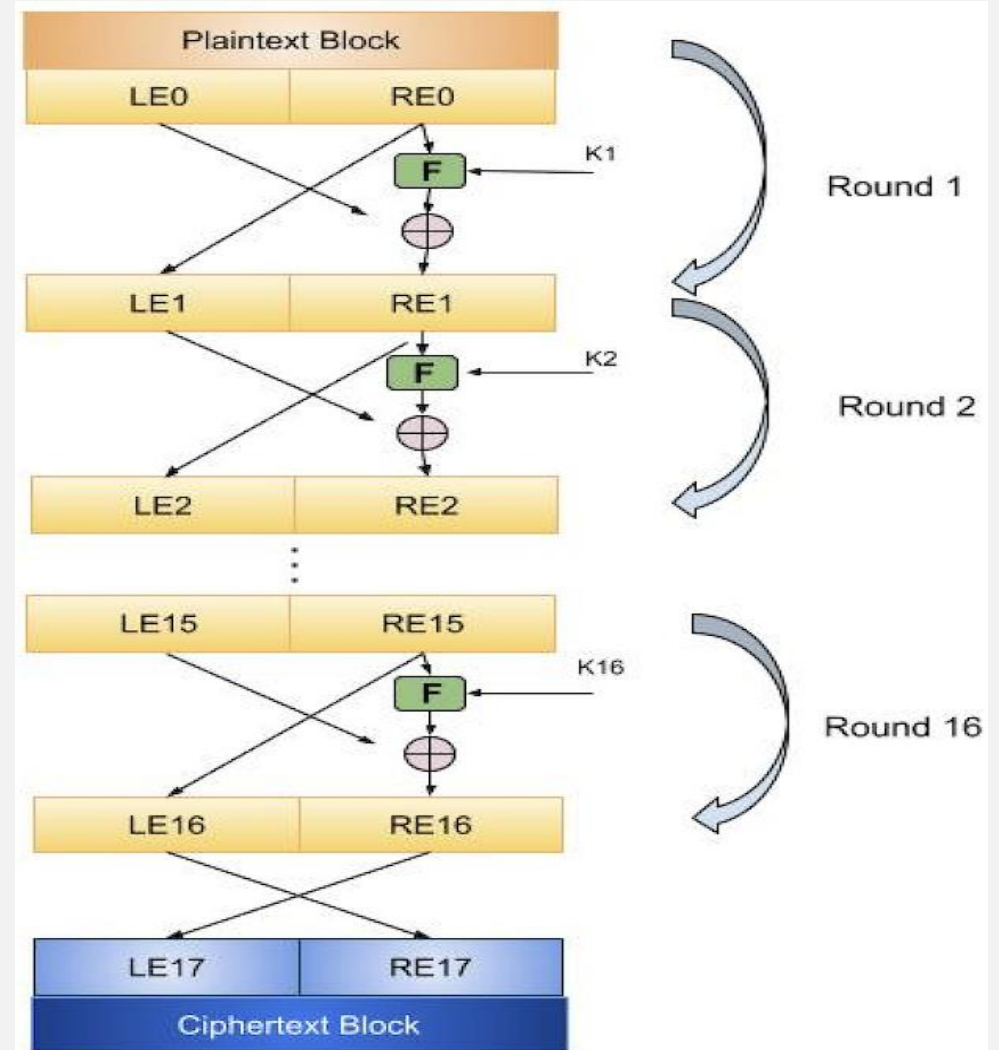


# DES...

## Step-3.5 XOR with Left Half 32 bit

- After P-box Permutation the 32-bit output is generated. It completes **Round Function  $f(R,K)$**
- Now the output is XORed with Left half of the 32-bit.

This whole process is repeated for 16 rounds.



# DES...

## Step-4 Swap L16 and R16

- At the end of round 16, the block is split into:

$$L_{16}(\text{left half}) \longleftrightarrow R_{16}(\text{right half})$$

## Step-5 Final Permutation

- After Swapping the final permutation takes place.
- Final Permutation table is referred for this step.
- It takes a **64-bit block** and rearranges bits using a fixed table.
- It produces the **final ciphertext (64-bit output)**
- It does NOT change bit values. It only changes bit positions.

# DES...

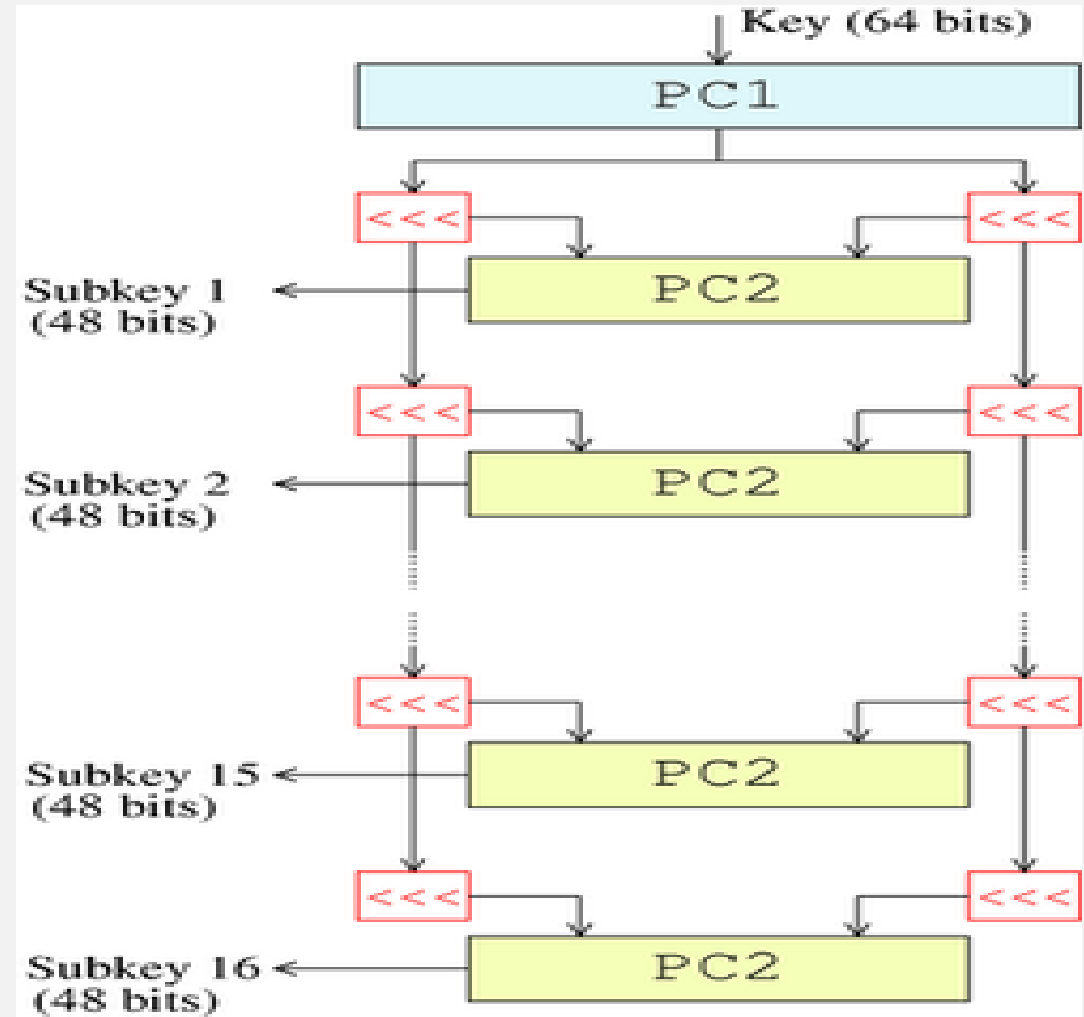
- Step-5 Final Permutation...

|    | Final Permutation table |    |    |    |    |    |    |
|----|-------------------------|----|----|----|----|----|----|
| 40 | 8                       | 48 | 16 | 56 | 24 | 64 | 32 |
| 39 | 7                       | 47 | 15 | 55 | 23 | 63 | 31 |
| 38 | 6                       | 46 | 14 | 54 | 22 | 62 | 30 |
| 37 | 5                       | 45 | 13 | 53 | 21 | 61 | 29 |
| 36 | 4                       | 44 | 12 | 52 | 20 | 60 | 28 |
| 35 | 3                       | 43 | 11 | 51 | 19 | 59 | 27 |
| 34 | 2                       | 42 | 10 | 50 | 18 | 58 | 26 |
| 33 | 1                       | 41 | 9  | 49 | 17 | 57 | 25 |

# DES KEY TRANSFORMATION

## DES Key Scheduling Steps:

1. Start with a **64-bit key**
2. Remove parity bits → **56-bit key**
3. Divide into:
  - $C_0$  (28 bits)
  - $D_0$  (28 bits)
4. For each round (1–16):
  - i. Perform left shifts on C and D (1 or 2 bits)
  - ii. Get  $C_i$  and  $D_i$
  - iii. Combine → 56-bit ( $C_iD_i$ )
  - iv. Apply **PC-2**:
    - Select and rearrange **48 bits**
    - Generate subkey  $K_i$
5. Repeat for all 16 rounds



# DES KEY TRANSFORMATION

- Perform left shifts on C and D (1 or 2 bits)

|                   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
|-------------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| Round             | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| #key bits shifted | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 2  | 2  | 2  | 2  | 2  | 2  | 1  |

Figure - number of key bits shifted per round

- After an appropriate shift, 48 of the 56 bit are selected. For selecting 48 of the 56 bits the table show in figure given below.

|    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|
| 14 | 17 | 11 | 24 | 1  | 5  | 3  | 28 | 15 | 6  | 21 | 10 |
| 23 | 19 | 12 | 4  | 26 | 8  | 16 | 7  | 27 | 20 | 13 | 2  |
| 41 | 52 | 31 | 37 | 47 | 55 | 30 | 40 | 51 | 45 | 33 | 48 |
| 44 | 49 | 39 | 56 | 34 | 53 | 46 | 42 | 50 | 36 | 29 | 32 |

Figure - compression permutation

# LIMITATIONS OF DES

## 1. Small Key Size (56-bit)

- DES uses only a **56-bit key**, which is too short by modern standards.
- Attackers can perform a **brute-force attack** by trying all possible keys.
- Today's computers can break DES in a very short time.

## 2. Vulnerable to Brute Force Attacks

- With advances in computing power, DES can be cracked easily.
- Example: The Electronic Frontier Foundation built a machine in 1998 that broke DES in less than 3 days.

## 3. Weak Against Differential Cryptanalysis

- DES is somewhat vulnerable to **Differential Cryptanalysis**.
- Though it was designed to resist it, the protection is not strong enough by modern standards.

## 4. Weak Keys Problem

- Some DES keys are considered **weak or semi-weak**.
- These keys produce repeated patterns, reducing security.

## 5. Limited Block Size (64-bit)

- DES operates on **64-bit blocks**, which is relatively small.
- This can lead to patterns in large data (birthday attacks).

# ADVANCE ENCRYPTION STANDARD

- Advanced Encryption Standard (AES) is a symmetric key block cipher widely used to secure data in computers, networks, and communication systems.
- It was selected by the U.S. National Institute of Standards and Technology (NIST) in 2001 to replace the older DES algorithm because it is faster, more secure, and more efficient.

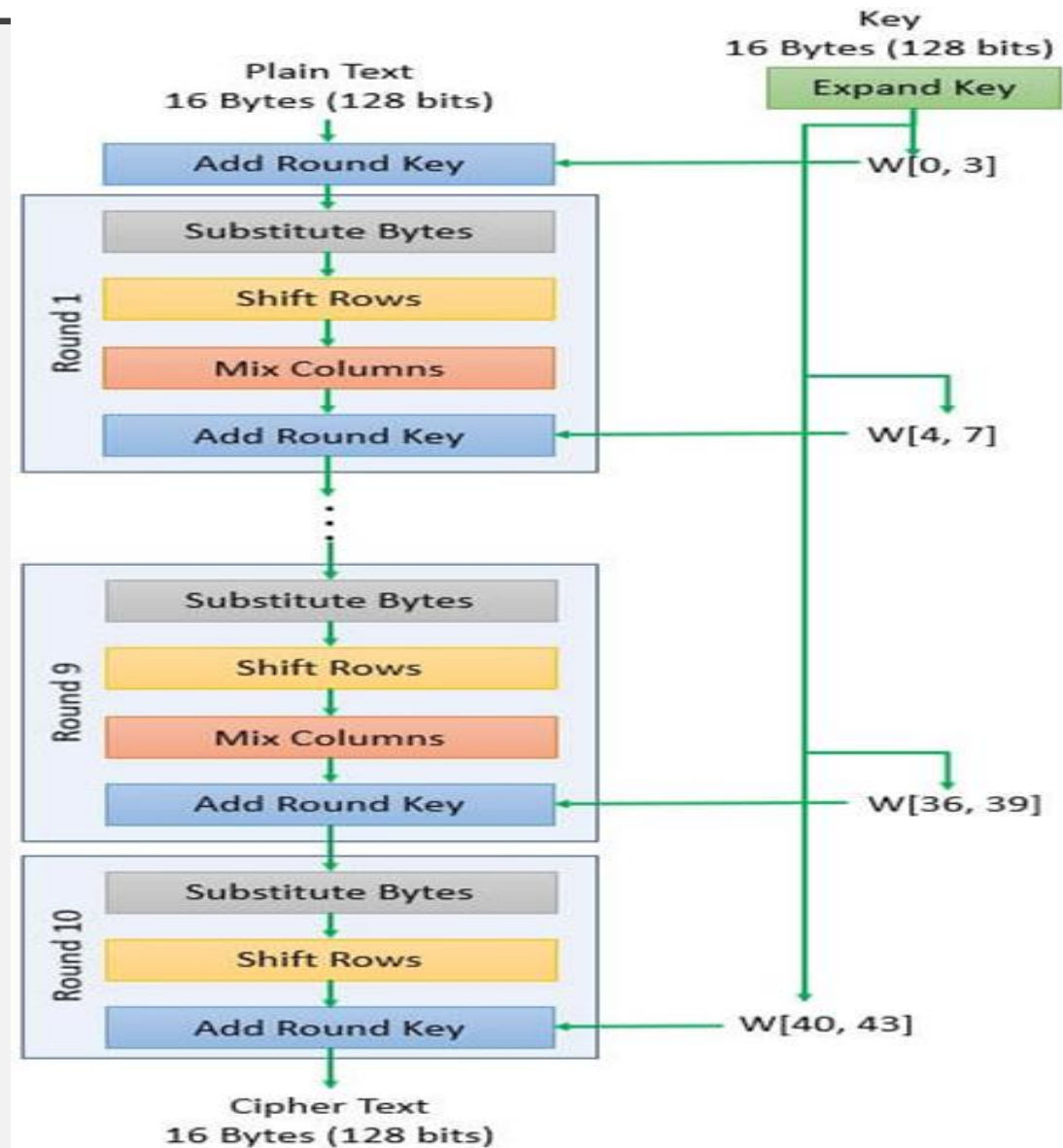
## Basic Idea of AES

- AES encrypts data in fixed-size blocks using the same secret key for both encryption and decryption.
- **Block size:** 128 bits (fixed)
- **Key sizes:** 128, 192, or 256 bits
- **Structure:** Substitution–Permutation Network (SPN), not Feistel like DES

# AES...

- Unlike DES, the number of rounds in AES is variable and depends on the length of the key.
- AES uses 10 rounds for 128-bit keys, 12 rounds for 192-bit keys and 14 rounds for 256-bit keys.
- Each of these rounds uses a different 128-bit round key, which is calculated from the original AES key.

| Rounds | Key Size |
|--------|----------|
| 10     | 128      |
| 12     | 192      |
| 14     | 256      |

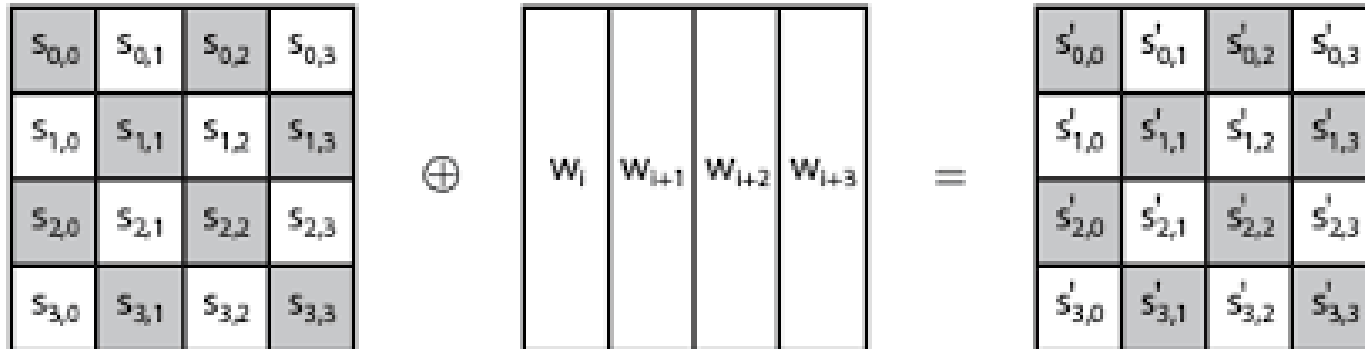




# AES ...

## KEY TRANSFORMATION

- AES works with **words** of 4 bytes (32 bits).
- 1 word = 4 bytes
- AES-128 bit key (16 bytes) = **4 words**  $\rightarrow W_0, W_1, W_2, W_3$
- We must generate words up to:  
AES-128  $\rightarrow$  **44 words** ( $4 \times 11$  round keys)



# AES....

## Step -I Byte Substitution (Sub Bytes)

- The 16 input bytes are substituted by looking up a fixed table (S-box) given in design. The result is in a matrix of four rows and four columns.

$$\text{State} = \begin{bmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{bmatrix}$$

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
| 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
| 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
| 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
| 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
| 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
| 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
| 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
| 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
| 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
| A | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
| B | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
| C | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
| D | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
| E | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
| F | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

# AES...

## STEP-2 SHIFT ROWS

- Each of the four rows of the matrix is shifted to the left. Any entries that 'fall off' are re-inserted on the right side of row. Shift is carried out as follows —
- **Input Values**

$$\begin{bmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{bmatrix}$$

## AFTER SHIFT ROWS

- Row 0: no change
- Row 1: shift left by 1 →  
55 99 *DD* 11
- Row 2: shift left by 2 →  
*AA EE* 22 66
- Row 3: shift left by 3 →  
*FF 33 77* *BB*

$$\begin{bmatrix} 00 & 44 & 88 & CC \\ 55 & 99 & DD & 11 \\ AA & EE & 22 & 66 \\ FF & 33 & 77 & BB \end{bmatrix}$$

# AES...

## Step-3 Mix Columns

- **Mix Columns** transforms **each column** (4 bytes) of the state matrix using a fixed mathematical rule.
- The 4 bytes of a column are taken as input
- They are combined to produce **4 new bytes**
- These 4 new bytes **replace** the original column
- After all columns are processed, we get a **new 4×4 matrix (16 new bytes)**

$$\begin{bmatrix} 00 & 44 & 88 & CC \\ 55 & 99 & DD & 11 \\ AA & EE & 22 & 66 \\ FF & 33 & 77 & BB \end{bmatrix}$$

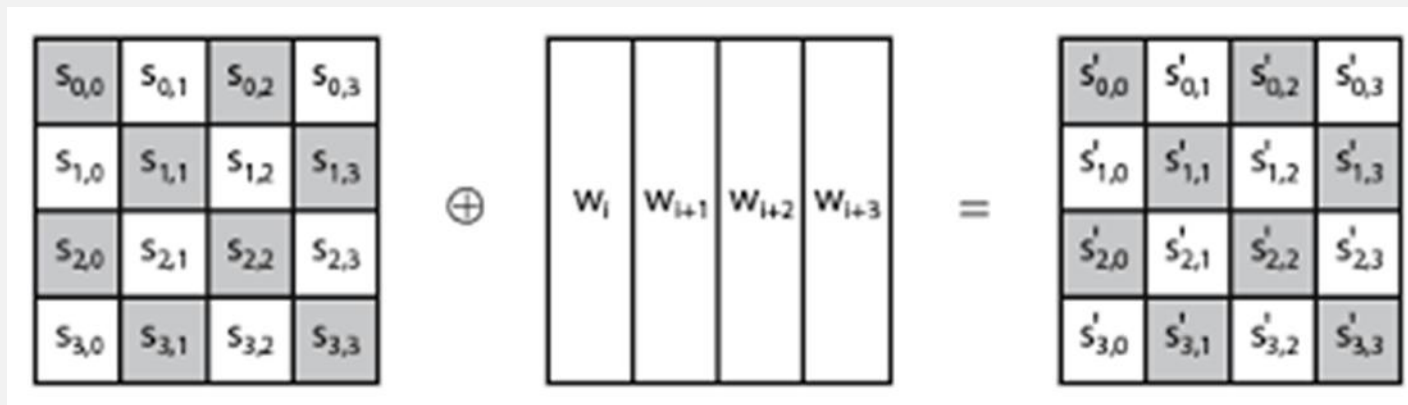
**X**

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

# AES

## Step-4 ADD ROUND KEY

- **Add Round Key** is the step where the **state matrix** is combined with a **round key**.
- The 4×4 state (16 bytes) is **XORed** with the 4×4 round key (16 bytes)
- XOR is done **byte by byte**, position by position



# AES

- These steps are repeated **10** times(For 128 bit).
- On the last round “**Mix Columns**” step is skipped.

# DECRYPTION PROCESS

- The process of decryption of an AES ciphertext is similar to the encryption process in the reverse order. Each round consists of the four processes conducted in the reverse order.
  - i. Add round key
  - ii. Mix columns
  - iii. Shift rows
  - iv. Byte substitution
- Since sub-processes in each round are in reverse manner, unlike for a Feistel Cipher, the encryption and decryption algorithms need to be separately implemented, although they are very closely related.

# NUMBER THEORY

Basic Concepts of Fields (Groups, Rings, Fields, Modular Arithmetic, Galois Fields, Polynomial Arithmetic)

2.4. Number Theory: Prime Numbers, Fermat's Theorem, Miller-Rabin Algorithm, Euclidean and Extended Euclidean Theorem, Euler Totient Function

2.5. Asymmetric Encryption: Diffie-Hellman Key Exchange, RSA Algorithm



# GROUP

A **group**  $\{G, \bullet\}$  is a set  $G$  together with a **binary operation**  $\bullet$  such that for all  $a, b, c \in G$ :

i. **(A1) Closure**

If  $a, b \in G$ , then  $a \bullet b \in G$ .

ii. **(A2) Associativity**

$$a \bullet (b \bullet c) = (a \bullet b) \bullet c$$

iii. **(A3) Identity element**

There exists an element  $e \in G$  such that:

$$a \bullet e = e \bullet a = a \text{ for all } a \in G.$$

iv. **(A4) Inverse element**

For each  $a \in G$ , there exists an element  $a^{-1} \in G$  such that

$$a \bullet a^{-1} = a^{-1} \bullet a = e.$$

**Note:** The symbol ' $\bullet$ ' is generic. It can represent addition, multiplication, or any binary operation.  
In simple words - A **Group** is a set of numbers with **only ONE operation** (usually  $+$  or  $\times$ ) that behaves nicely.

# GROUP

## Example: Integers with addition $\rightarrow (\mathbb{Z}, +)$

- Take numbers:  $\dots, -2, -1, 0, 1, 2, \dots$
- Operation: **addition (+)**
- Check the rules:
  - Closure:**  $2 + 3 = 5$  (still an integer) ✓
  - Associative:**  $(2 + 3) + 4 = 2 + (3 + 4)$  ✓
  - Identity:**  $0 \rightarrow 5 + 0 = 5$  ✓
  - Inverse:** For 5, inverse is  $-5 \rightarrow 5 + (-5) = 0$  ✓
- So  $(\mathbb{Z}, +)$  is a **group**.
- **Abelian Group:**  $2+3=3+2$  ✓

# GROUP...

**Abelian Group** : A group is **abelian** if it also satisfies:

v. (A5) Commutativity

$$a \bullet b = b \bullet a \text{ for all } a, b \in G.$$

*In simple: An **Abelian group** is a group where **order doesn't matter**:*

**Cyclic Group**: A group  $G$  is **cyclic** if there exists an element  $a \in G$  such that **every element** of  $G$  can be written as:

$$a^k \text{ for some integer } k.$$

- The element  $a$  is called a **generator** of the group.
- *In Simple: A **Cyclic group** is a group where **every element can be generated by repeatedly applying the operation to a single element** (called a generator).*
- **Example:  $(\{0, 1, 2, 3\}, + \text{ mod } 4)$**

**Notes:** Every cyclic group is abelian.

# RINGS

- A ring  $R$ , denoted by  $\{R, +, \times\}$ , is a set equipped with two binary operations (addition and multiplication) such that for all  $a, b, c \in R$ :
  - i. **(A1–A5) Addition axioms:**  
 $(R, +)$  is an **abelian group**.
  - ii. **(M1) Closure under multiplication:**  
If  $a, b \in R$ , then  $ab \in R$ .
  - iii. **(M2) Associativity of multiplication:**  
 $a(bc) = (ab)c$ .
  - iv. **(M3) Distributive laws:**  
 $a(b + c) = ab + ac$  and  $(a + b)c = ac + bc$ .

# RING

- **Example: Integers with  $+$  and  $\times \rightarrow (\mathbb{Z}, +, \times)$**
- $2 + 3 = 5$  ✓
- $2 \times 3 = 6$  ✓
- Addition forms a group (as above) ✓
- But division problem:  $3 \div 2 = 1.5$  ✗ (not an integer)
- So, this is a **Ring**, not a Field.

# RINGS...

## Commutative Ring

A ring  $R$  is said to be **commutative** if it additionally satisfies:

- **(M4) Commutativity of multiplication:**

$$ab = ba \text{ for all } a, b \in R.$$

## Integral Domain

A ring  $R$  is called an **integral domain** if it satisfies all of the following conditions:

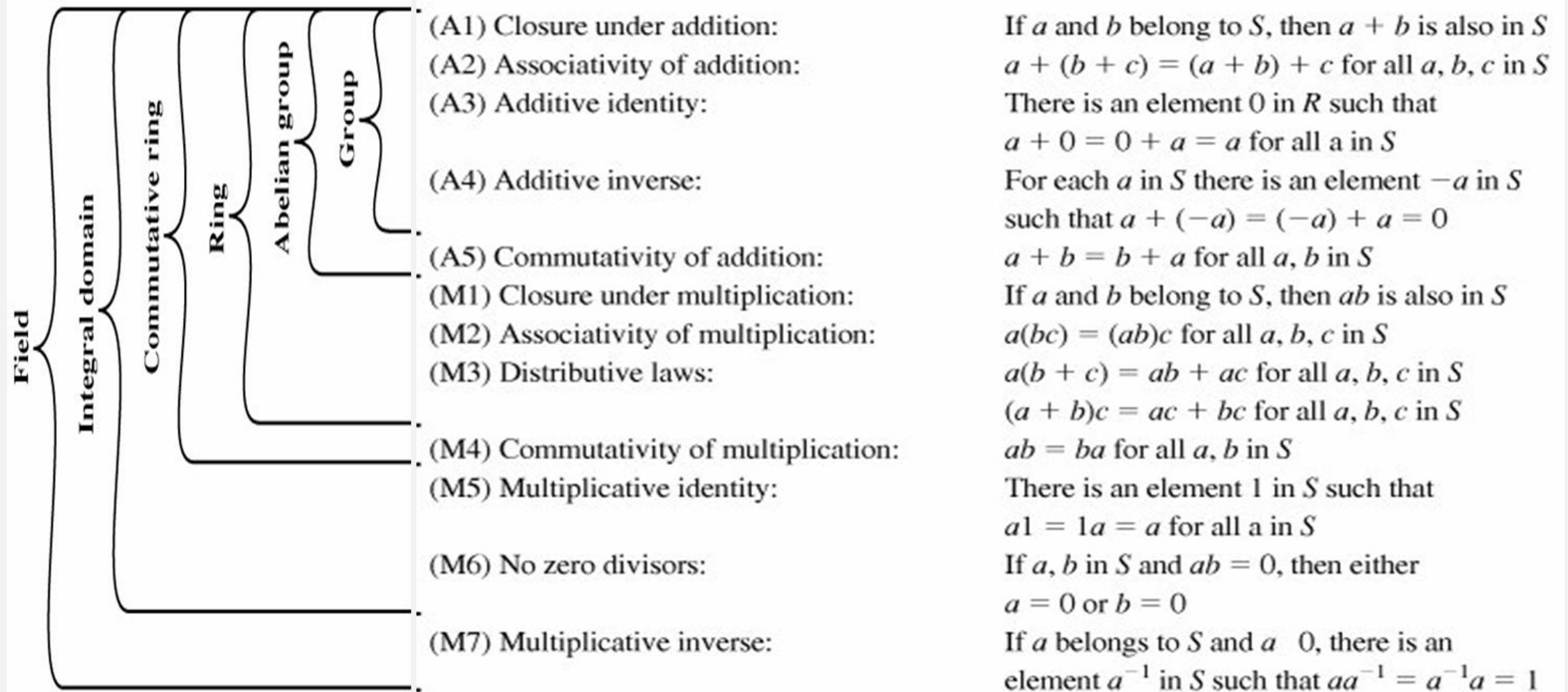
- $R$  is a **commutative ring**.
- (M5) Multiplicative identity:** There exists an element  $1 \in R$  such that  $a \cdot 1 = 1 \cdot a = a$  for all  $a \in R$ .
- (M6) No zero divisors:** If  $ab = 0$  for  $a, b \in R$ , then  $a = 0$  or  $b = 0$ .

**Definition:** An integral domain is a **commutative ring with unity and no zero divisors**.

# FIELDS

- A field  $F$ , denoted by  $\{F, +, \times\}$ , is a set equipped with two binary operations, **addition** and **multiplication**, such that for all  $a, b, c \in F$ :
- **(A1–M6) Integral Domain:**  $F$  is an **integral domain**, i.e., it satisfies:
  - i.  $(F, +)$  is an **abelian group** (A1–A5)
  - ii. Multiplication is **closed** and **associative** (M1–M2)
  - iii. Distributive laws hold (M3)
  - iv. Multiplication is **commutative** (M4)
  - v. There exists a **multiplicative identity**  $1 \in F$  (M5)
  - vi. **No zero divisors** exist (M6)
- **(M7) Multiplicative inverse:** For each  $a \in F, a \neq 0$ , there exists an element  $a^{-1} \in F$  such that:
$$a \cdot a^{-1} = a^{-1} \cdot a = 1$$
- **Definition :** A field is a **commutative ring with unity** in which **every nonzero element has a multiplicative inverse**.

# SUMMARY





# GCD (GREATEST COMMON DIVISOR)

- The **GCD** of two numbers is **the largest number that can divide both numbers exactly** (without leaving a remainder).

## Examples: GCD of 12 and 18

- Factors of 12: 1, 2, 3, 4, 6, 12
- Factors of 18: 1, 2, 3, 6, 9, 18
- Common factors: 1, 2, 3, 6 → **Largest is 6**
- $\text{GCD}(12, 18) = 6$

# GALOIS FIELD

- “A **Galois Field** is a set of numbers where you can **add, subtract, multiply, and divide (except by 0)** and the result always stays within the set.”
- **Example 2:**  $\text{gf}(5) = (0, 1, 2, 3, 4)$  which consists of 5 elements where each of them is a polynomial of degree 0 (a constant) while
- $\text{gf}(2^3) = (0, 1, 2, 3, 4, 5, 6, 7)$  which consists of  $2^3 = 8$  elements where each of them is a polynomial of degree at most 2 evaluated at 2.

■

# MODULAR ARITHMETIC

- When we divide a number by another positive number, we get a **quotient** and a **remainder**. Working with these **remainders** is called **modular arithmetic**. This idea is very important in mathematics and is widely used in computer science.
- In modular arithmetic, we only use a **limited set of results**. For example, a clock has only **12 hours**. If the time is 7 o'clock now, then **20 hours later** it will be **3 o'clock**, not 27 o'clock. Because after 12, the numbers start again from 1. This is why modular arithmetic is often called **clock arithmetic**.
- **Example:**  
If we do calculations **modulo 12** (like a 12-hour clock), the numbers start again after reaching 12. They keep repeating in a cycle, just like the hours on a clock.

# MODULAR ARITHMETIC

- **Addition:** Let's say we want to calculate:  $7+9$  modulo 12.  
$$= (7+9) \text{ modulo } 12$$
$$= 16 \bmod 12 = 4$$
- **Subtraction:** Now, let's calculate:  $5-8$  modulo 12  
$$= (5-8) \bmod 12$$
$$= -3 \bmod 12$$
  - Since -3 is negative, we add 12 until we get a positive result:  $-3+12=9$
  - So,  $9 \bmod 12 = 9$

# CONGRUENCE MODULO

- Congruence modulo  $n$  is a fundamental concept in modular arithmetic. It indicates that two numbers  $a$  and  $b$  have the same remainder, denoted as  $a \equiv b \pmod{n}$ .
- Two numbers are **congruent modulo**  $n$  if they give the **same remainder** when divided by  $n$ .
- Formally,  $a \equiv b \pmod{n}$  if  $n$  divides  $a-b$ , which is expressed as  $a-b = kn$  for some integer  $k$ .
- **For example:**
  - $17 \equiv 5 \pmod{6}$  because  $17-5=12$  is divisible by 6.
  - $11 \equiv -1 \pmod{12}$  because  $11-(-1)=12$  is divisible by 12.
  - $23 \equiv 5 \pmod{9}$  because  $23-5=18$  is divisible by 9.

# ADDITIVE INVERSE

**Definition:**

In  $GF(p)$ , the **additive inverse** of a number  $a$  is the number which, when added to  $a$ , gives  $0 \pmod{p}$ .

$$a + (-a) \equiv 0 \pmod{p}$$

■ **Example in  $GF(7)$ :**

Additive inverse of 3 is 4 because  $3 + 4 = 7 \equiv 0$ .

## ADDITIVE INVERSE

✓ In  $\text{GF}(97)$

Let  $a = 68$

$$-a \equiv 97 - 68 = 29$$

$$\text{Check: } 68 + 29 = 97 \equiv 0$$

---

Let  $a = 145$  (larger than 97)

First reduce:

$$145 \bmod 97 = 48$$

Additive inverse:

$$97 - 48 = 49$$

$$\text{Check: } 145 + 49 = 194 \equiv 0 \pmod{97}$$

# MULTIPLICATIVE INVERSE

## Definition:

In  $\text{GF}(p)$ , the **multiplicative inverse** of a non-zero number  $a$  is the number which, when multiplied by  $a$ , gives 1 (mod  $p$ ).

$$a \times a^{-1} \equiv 1 \pmod{p}$$

- **Note:** 0 has **no** multiplicative inverse.

- **Example in  $\text{GF}(7)$ :**

Multiplicative inverse of 3 is 5 because  
 $3 \times 5 = 15 \equiv 1$ .



# MULTIPLICATIVE INVERSE

## ✓ In GF(7)

Find inverse of 3:

Try multiples of 3:

$$3 \times 1 = 3, 3 \times 2 = 6, 3 \times 3 = 9 \equiv 2, 3 \times 4 = 12 \equiv 5, 3 \times 5 = 15 \equiv 1$$

$$3^{-1} = 5$$

## ✓ In GF(11)

Find inverse of 7:

$$7 \times 8 = 56 \equiv 1 \pmod{11}$$

$$7^{-1} = 8$$

## ✓ In GF(13)

Find inverse of 5:

$$5 \times 8 = 40 \equiv 1 \pmod{13}$$

$$5^{-1} = 8$$

# MULTIPLICATION AND ADDITIVE INVERSES

| $\times$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|----------|---|---|---|---|---|---|---|---|
| 0        | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1        | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 2        | 0 | 2 | 4 | 6 | 0 | 2 | 4 | 6 |
| 3        | 0 | 3 | 6 | 1 | 4 | 7 | 2 | 5 |
| 4        | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 4 |
| 5        | 0 | 5 | 2 | 7 | 4 | 1 | 6 | 3 |
| 6        | 0 | 6 | 4 | 2 | 0 | 6 | 4 | 2 |
| 7        | 0 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |

(b) Multiplication modulo 8

| $w$ | $-w$ | $w^{-1}$ |
|-----|------|----------|
| 0   | 0    | —        |
| 1   | 7    | 1        |
| 2   | 6    | —        |
| 3   | 5    | 3        |
| 4   | 4    | —        |
| 5   | 3    | 5        |
| 6   | 2    | —        |
| 7   | 1    | 7        |

(c) Additive and multiplicative inverses modulo 8

# ORDINARY POLYNOMIAL ARITHMETIC

$$\begin{array}{r}
 x^3 + x^2 \quad + 2 \\
 + (x^2 - x + 1) \\
 \hline
 x^3 + 2x^2 - x + 3
 \end{array}$$

(a) Addition

$$\begin{array}{r}
 x^3 + x^2 \quad + 2 \\
 - (x^2 - x + 1) \\
 \hline
 x^3 \quad + x + 1
 \end{array}$$

(b) Subtraction

$$\begin{array}{r}
 x^3 + x^2 \quad + 2 \\
 \times (x^2 - x + 1) \\
 \hline
 x^3 + x^2 \quad + 2
 \end{array}$$

$$-x^4 - x^3 - 2x$$

$$x^5 + x^4 + 2x^2$$

$$\begin{array}{r}
 x^5 \quad + 3x^2 - 2x + 2
 \end{array}$$

(c) Multiplication

$$\begin{array}{r}
 x^2 - x + 1 \overline{) x^3 + x^2 + 2} \\
 \underline{x^3 + x^2 + x} \phantom{+ 2} \\
 2x^2 - x + 2 \\
 \underline{2x^2 - 2x + 2} \\
 x
 \end{array}$$

(d) Division

In GF(2),  
**addition** and  
**multiplicati**  
**on** are  
equivalent to  
the **XOR**,  
and the  
logical  
**AND**,  
respectively.  
Addition and  
subtraction  
are  
equivalent.  
Therefore  
GF(2<sup>n</sup>) is of  
most interest  
in.

$$\begin{array}{r}
 x^7 \qquad + x^5 + x^4 + x^3 \qquad + x + 1 \\
 \qquad \qquad \qquad + (x^3 \qquad + x + 1) \\
 \hline
 x^7 \qquad + x^5 + x^4
 \end{array}$$

(a) Addition

$$\begin{array}{r}
 x^7 \qquad + x^5 + x^4 + x^3 \qquad + x + 1 \\
 \qquad \qquad \qquad - (x^3 \qquad + x + 1) \\
 \hline
 x^7 \qquad + x^5 + x^4
 \end{array}$$

(b) Subtraction

$$\begin{array}{r}
 \begin{array}{r}
 x^7 \qquad + x^5 + x^4 + x^3 \qquad + x + 1 \\
 \qquad \qquad \qquad \times (x^3 \qquad + x + 1) \\
 \hline
 x^7 \qquad + x^5 + x^4 + x^3 \qquad + x + 1 \\
 x^8 \qquad + x^6 + x^5 + x^4 \qquad + x^2 + x \\
 \hline
 x^{10} \qquad + x^8 + x^7 + x^6 \qquad + x^4 + x^3 \\
 \hline
 x^{10} \qquad \qquad \qquad + x^4 \qquad + x^2 \qquad + 1
 \end{array}
 \end{array}$$

(c) Multiplication

$$\begin{array}{r}
 \begin{array}{r}
 x^4 + 1 \\
 \hline
 x^3 + x + 1 \overline{) x^7 + x^5 + x^4 + x^3 + x + 1} \\
 \underline{x^7 + x^5 + x^4} \phantom{+ x + 1} \\
 \phantom{x^7 + x^5 + x^4} x^3 \phantom{+ x^4} + x + 1 \\
 \phantom{x^7 + x^5 + x^4} \underline{x^3 \phantom{+ x^4} + x + 1} \\
 \phantom{x^7 + x^5 + x^4} \phantom{x^3 + x^4} 0
 \end{array}
 \end{array}$$

(d) Division

## EXAMPLE: POLYNOMIAL ARITHMETIC IN $\mathbb{Z}_p$

$$\begin{aligned} f(x) &= x^4 + x^2 + x + 1 \\ g(x) &= x^3 + x + 1 \end{aligned}$$

— Over  $\text{G.F.}(\mathbb{Z}_2) \Rightarrow \{0, 1\}$

$$\begin{aligned} f(x) + g(x) &= x^4 + x^3 + x^2 + x + x + 1 + 1 \\ &= x^4 + x^3 + x^2 + 2x + 2 \\ &= x^4 + x^3 + x^2 + 0 \cdot x + 0 \\ &= x^4 + x^3 + x^2 \end{aligned}$$

$$\begin{aligned} f(x) - g(x) &= x^4 + x^2 + x + 1 - (x^3 + x + 1) \\ &= x^4 + x^2 + x + 1 \\ &\quad + \underline{1x^3 + 0x + 0} \\ &= x^4 + x^3 + x^2 \end{aligned}$$

# MULTIPLICATION

$$f(x) * g(x) = (x^4 + x^2 + x + 1) (x^3 + x + 1)$$

$$= x^7 + \cancel{x^5} + \cancel{x^4} + \cancel{x^3} + \cancel{x^3} + x^2 + \cancel{x^4} + \cancel{x^2} + x + \cancel{x^3} + x + 1$$

$$= x^7 + 1$$

# DIVISION

Divide  $5x^2 + 4x + 6$  by  $2x + 1$  over  $\text{GF}(7)$

$$\begin{array}{r}
 2x+1 \overline{) 5x^2+4x+6} \\
 \underline{2x^2+x} \phantom{+6} \\
 3x^2+5x+6 \\
 \underline{3x^2+x+1} \phantom{+6} \\
 4x+5 \phantom{+6} \\
 \underline{4x+1} \phantom{+6} \\
 4 \phantom{+6}
 \end{array}$$

$4x + 5 - (4x + 1) = 4$

$\rightarrow 4$

"Divide  $3x^2 + 4x + 3$  by  $5x + 6$  over  $\text{GF}(7)$  or  $\mathbb{Z}_7$ ."

# PRIME NUMBERS

- A **prime number** is a whole number greater than **1** that can be divided **only** by **1** and itself.
- Examples: 2,3,5,7,11,13, ...
- The number **1** is neither prime nor composite.
- **2** is the smallest and the only even prime.
- Every other prime number is odd.



# ROLE OF PRIME NUMBERS IN CRYPTOGRAPHY

- Prime numbers are very important in cryptography because many security algorithms perform calculations **modulo a large prime number**. This makes the system mathematically strong and difficult to break.
- i. **Key Generation — RSA and Diffie–Hellman key exchange**
  - RSA creates public and private keys using very large prime numbers.
  - Diffie–Hellman uses a large prime number to securely exchange keys.
- ii. **Primality Testing — Miller–Rabin primality test**
  - Before using a number, cryptographic systems test whether it is prime. Miller–Rabin is a fast method for this purpose.
- iii. **Digital Signatures — Digital Signature Algorithm**
  - Digital signatures depend on calculations done with large prime numbers to ensure authenticity.
- iv. **Random Number Generation**
  - Certain secure random number generators use special prime numbers to produce unpredictable values.
- v. **Elliptic Curve Cryptography — Elliptic Curve Cryptography**
  - Elliptic curves are commonly defined over fields formed using large prime numbers.
- vi. **Hash Functions**
  - Some cryptographic hash functions use prime numbers in their design to improve security and distribution.

# FERMAT'S THEOREM

- Fermat's Little Theorem states that for any prime number  $p$  and any integer  $a$ , the following equation holds:

$$a^p \equiv a \pmod{p} \quad \text{or,} \quad a^{p-1} \equiv 1 \pmod{p}$$

- where  $\equiv$  means congruent modulo  $p$ , and  $\pmod{p}$  indicates that we are working modulo  $p$ .
- In simpler terms, Fermat's Little Theorem tells us that if we raise any number  $a$  to the power of a prime number  $p$  and then divide the result by  $p$ , the remainder will always be the same as if we had divided  $a$  by  $p$ .
- For example, let's take  $a = 3$  and  $p = 5$ . We can verify that the theorem holds by computing:  
 $3^5 = 243 \equiv 3 \pmod{5}$  ---which means that if we divide 243 by 5, the remainder is 3.  
Similarly, if we divide 3 by 5, the remainder is also 3.
- Fermat's Little Theorem is a powerful tool in number theory and cryptography

# MILLER–RABIN PRIMALITY TEST

- Miller–Rabin is a **fast test** to check whether a number is **probably prime**.
- It does **not** prove a number is prime with 100% certainty, but after a few rounds, the chance of mistake becomes **extremely small**.

## Idea behind the test

- It is based on **Fermat's Little Theorem**:
- If  $p$  is prime, then

$$a^{p-1} \equiv 1 \pmod{p}$$

- But some composite numbers can also pass this (called Fermat liars). Miller–Rabin adds extra checks to catch those.

# MILLER-RABIN...

## Steps

- Let the number be  $n$ .

### 1) Write

$$n - 1 = 2^r \times d \text{ (where } d \text{ is odd)}$$

### 2) (Pick any number $a$ such that

$$2 \leq a \leq n - 2$$

### 3) Compute

$$x = a^d \bmod n$$

### 4) Check

If  $x = 1$  or  $x = n - 1 \rightarrow$  **pass**

Else, keep doing:

$$x = x^2 \bmod n$$

up to  $r - 1$  times

- If you get  $n - 1 \rightarrow$  **pass**
- If you never get  $n - 1 \rightarrow$  **composite**

# MILLER-RABIN...

- **Step 0 — Given**  $n = 25$
- **Choose base**  $a = 2$      **Check:**  $2 \leq a \leq n - 2$
- **Step 1 — Write**  $n - 1 = 2^r \times d$ 
  - Factor out powers of 2:  $(n-1)=24$   
 $24 = 2^3 \times 3$
  - So,  $r = 3, d = 3$  (odd)
- **Step 2 — Compute**  $x = a^d \bmod n$ 
  - $x = 2^3 \bmod 25 = 8$
- **Check:**
  - Is  $x = 1$ ? ❌ Is  $x = n - 1 = 24$ ? ❌
  - So, we go to the squaring loop.

- **Step 3 — Squaring loop (repeat  $r - 1 = 2$  times)**
- **First squaring**
  - $x = 8^2 \bmod 25 = 64 \bmod 25 = 14$
  - Check:  $x = 24$ ? ❌
- **Second squaring**
  - $x = 14^2 \bmod 25 = 196 \bmod 25 = 21$
  - Check:  $x = 24$ ? ❌

Since,  $x \neq 1$ ,  $x \neq n - 1$  ..  $n$  is composite number.

Let's run the Miller–Rabin Primality Test for

$$n = 571$$

Choose base  $a = 2$ .

**Step 1 — Write  $n - 1 = 2^r \times d$**

$$n - 1 = 570$$

Factor out powers of 2:

$$570 = 2 \times 285$$

So,

$$r = 1, \quad d = 285 \text{ (odd)}$$

**Multiply only the needed ones**

We need:

$$2^{256}, 2^{16}, 2^8, 2^4, 2^1$$

So multiply modulo 571:

1.  $230 \times 460 \equiv 108$
2.  $108 \times 256 \equiv 433$
3.  $433 \times 16 \equiv 87$
4.  $87 \times 2 \equiv 174$

$$x = 174$$

**Miller–Rabin check (since  $r = 1$ )**

Here  $n - 1 = 570$ .

We got:

$$x = 174 \neq 1, \quad x \neq 570$$

Since  $r - 1 = 0$ , there is **no squaring loop**.

**Conclusion for base  $a = 2$**

571 is composite for base 2

(So base 2 is a **witness**.)

# EUCLIDEAN THEOREM

- The **Euclidean Algorithm** is a **systematic method** to find the **Greatest Common Divisor (GCD)** of two integers.
- **GCD (Greatest Common Divisor)** of two numbers is the **largest number** that divides both of them without leaving a remainder.
- The Euclidean Algorithm is **efficient** because it reduces the problem into smaller numbers using **repeated division**.

## Step-by-Step Process to Find GCD (Euclidean Algorithm)

- To find the **Greatest Common Divisor (GCD)** of two numbers  $a$  and  $b$ :
- Identify the numbers:** Let  $r1$  be the larger number and  $r2$  be the smaller number.
- Divide:** Divide  $r1$  by  $r2$  and find the remainder  $r$ .
- $r1 \div r2 \Rightarrow \text{remainder } r$
- Shift the values :** Replace:
  - $r1$  with  $r2$
  - $r2$  with  $r$
- Repeat the process:** Continue dividing and shifting until the remainder becomes 0.
- Final Result:** The **last non-zero remainder** obtained in the process is the **GCD** of the two numbers.

# EUCLIDEAN THEOREM

Find  $\text{GCD}(48, 18)$

| Step | $q$ | $r_1$ | $r_2$ | $r$ |
|------|-----|-------|-------|-----|
| 1    | 2   | 48    | 18    | 12  |
| 2    | 1   | 18    | 12    | 6   |
| 3    | 2   | 12    | 6     | 0   |



- Here,  $r_1 \div r_2$  gives quotient  $q$  and remainder  $r$ .
- Replace  $r_1 \leftarrow r_2$  and  $r_2 \leftarrow r$  each step.

**Result:** The last non-zero remainder is 6.



# GCD(101,23)

## Solution (Tabular Form)

| Step | $q$ | $r_1$ | $r_2$ | $r$ |
|------|-----|-------|-------|-----|
| 1    | 4   | 101   | 23    | 9   |
| 2    | 2   | 23    | 9     | 5   |
| 3    | 1   | 9     | 5     | 4   |
| 4    | 1   | 5     | 4     | 1   |
| 5    | 4   | 4     | 1     | 0   |



**Result:** The last non-zero remainder is 1.

$$\text{GCD}(101, 23) = 1$$

# EXTENDED EUCLIDEAN THEOREM

- **Extended Euclidean Algorithm**
- The **Extended Euclidean Algorithm (EEA)** is an extension of the Euclidean Algorithm for GCD.

While the Euclidean Algorithm finds:

$$\gcd(a, b)$$

- the **Extended** version finds integers  $s$  and  $t$  such that:

$$as + bt = \gcd(a, b)$$

- These  $s$  and  $t$  are called **Bézout coefficients**.

$$s = s_1 - s_2 * q \quad \text{where } s_1 = 1 \text{ and } s_2 = 0$$

$$t = t_1 - t_2 * q, \quad t_1 = 0 \text{ and } t_2 = 0$$

# USE EXTENDED EUCLIDEAN ALGORITHM FOR (11, 3)

- Initialize:  $r_1 = 11, r_2 = 3, s_1 = 1, s_2 = 0, t_1 = 0, t_2 = 1$

| q | $r_1$ | $r_2$ | r | $s_1$ | $s_2$ | s  | $t_1$ | $t_2$ | t   |
|---|-------|-------|---|-------|-------|----|-------|-------|-----|
| - | 11    | 3     | - | 1     | 0     | -  | 0     | 1     | -   |
| 3 | 11    | 3     | 2 | 1     | 0     | 1  | 0     | 1     | -3  |
| 1 | 3     | 2     | 1 | 0     | 1     | -1 | 1     | -3    | 4   |
| 2 | 2     | 1     | 0 | 1     | -1    | 3  | -3    | 4     | -11 |

**Step 2:** Take coefficients from the row where  $r = 1$

$$s = -1, t = 4$$

This means:

$$11(-1) + 3(4) = 1$$

**Step 3: Identify the inverse:** The coefficient of **3** is 4. So,

$$3 \cdot 4 \equiv 1 \pmod{11}$$

**Multiplicative Inverse:**  $3^{-1} \equiv 4 \pmod{11}$

# EXTENDED EUCLIDEAN THEOREM

| q | r1  | r2 | r | s1 | s2 | s  | t1 | t2  | t   |
|---|-----|----|---|----|----|----|----|-----|-----|
| - | 101 | 23 | - | 1  | 0  | -  | 0  | 1   | -   |
| 4 | 101 | 23 | 9 | 1  | 0  | 1  | 0  | 1   | -4  |
| 2 | 23  | 9  | 5 | 0  | 1  | -2 | 1  | -4  | 9   |
| 1 | 9   | 5  | 4 | 1  | -2 | 3  | -4 | 9   | -13 |
| 1 | 5   | 4  | 1 | -2 | 3  | -5 | 9  | -13 | 22  |

## Initialize

Let  $r_1 = 101, r_2 = 23$

Let  $s_1 = 1, s_2 = 0$  for  $r_1$  and  $r_2$  (

Let  $t_1 = 0, t_2 = 1$  for  $r_1$  and  $r_2$  (

The formula for each step:

$$s = s_1 - q \cdot s_2 \quad \text{and} \quad t = t_1 - q \cdot t_2$$

## Final results from table

- $\gcd(101, 23) = 1$
- Bézout form:  $1 = -5(101) + 22(23)$
- Multiplicative inverse of 23 mod 101 is 22.

## QUESTION:

$$A(x) = x^5 + x^2 + 1, B(x) = x^4 + x + 1$$

- Use the **Extended Euclidean Algorithm over GF(2)** to find the Multiplicative Inverse

# EULER'S TOTIENT FUNCTION ( $\Phi$ )

- **Euler's Totient Function**, denoted  $\phi(n)$ , counts how many positive integers  $\leq n$  are **coprime** with  $n$  (i.e., their GCD with  $n$  is 1).
- Two numbers are coprime if  $\gcd(a, n) = 1$ .

## Formula

- If  $n$  is prime :  $\phi(n) = n - 1$
- If  $n$  is multiple of two primes  $p$  and  $q$ :  $\phi(n) = (p - 1)(q - 1)$
- If  $n$  is the power of prime  $p$ :  $\phi(n) = p^k - p^{k-1}$
- If  $n$  is the product of more than two primes:

$$\phi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \dots$$

# EULER TOTIENT FUNCTION

- ◆ Example 1: Prime  $n = 23$

$$\varphi(23) = 23 \left(1 - \frac{1}{23}\right) = 22$$

- ◆ Example 2: Prime power  $n = 16 = 2^4$

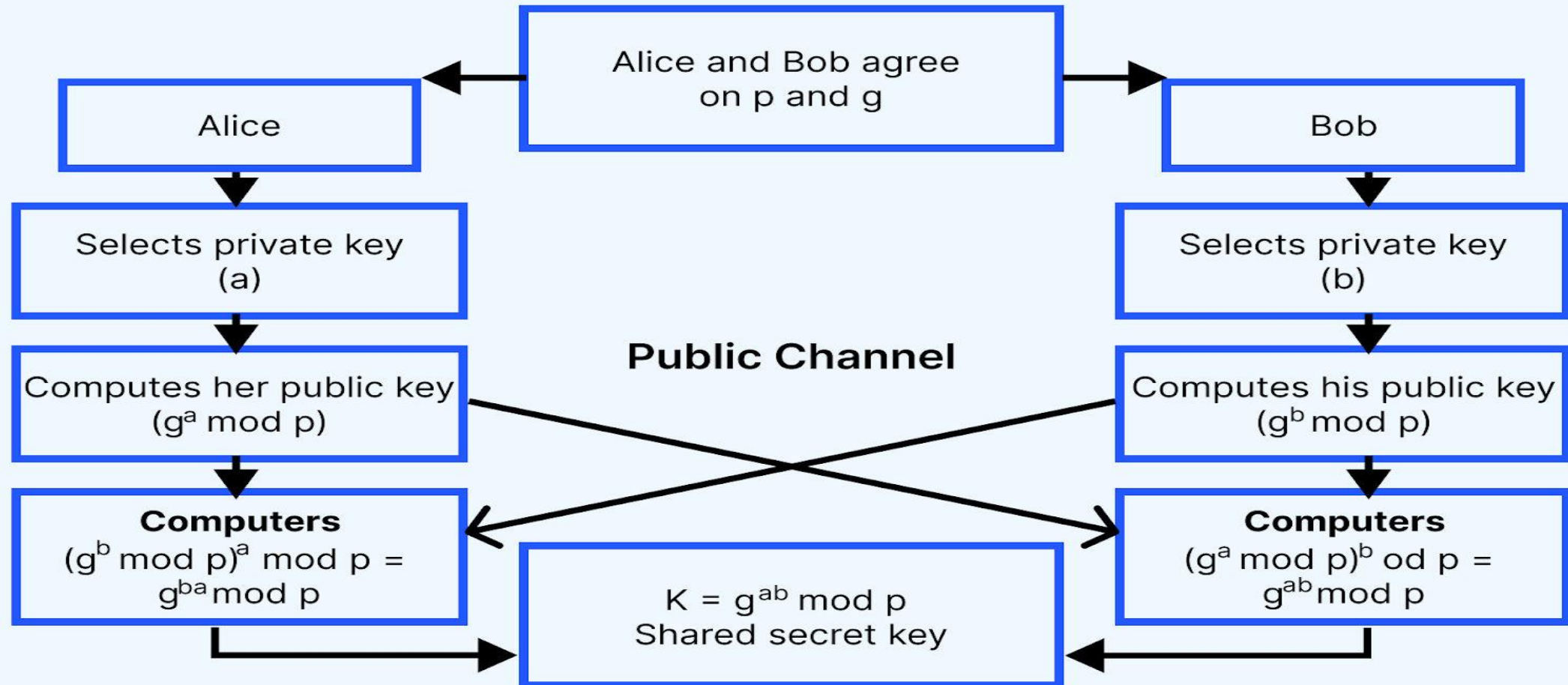
$$\varphi(16) = 16 \left(1 - \frac{1}{2}\right) = 8$$

- ◆ Example 3: Mixed factors  $n = 60 = 2^2 \times 3 \times 5$

$$\varphi(60) = 60 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{3}\right) \left(1 - \frac{1}{5}\right) = 16$$

# DIFFIE-HELLMAN KEY EXCHANGE

$p$  is prime number and  $g$  is generator





# DIFFIE-HELLMAN....

Both Alice and Bob agree on  $p=11$  and  $g=2$ , 'g' is primitive root of prime 'p'

| Alice                                                                                                                                       | Bob                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Selects private key( $a = 5$ )                                                                                                           | 1. Selects private key( $b = 7$ )                                                                                                                          |
| 2. Computes public key( $A = g^a \bmod p$ )<br>$A = 2^5 \bmod 11 = 32 \bmod 11 = 10$                                                        | 2. Computes public key( $B = g^b \bmod p$ )<br>$B = 2^7 \bmod 11 = 128 \bmod 11 = 7$                                                                       |
| 3. Key Exchange (over public channel)<br>Sends <b>A = 10</b> →                                                                              | 3. Key Exchange (over public channel)<br>← Sends <b>B = 7</b>                                                                                              |
| 4. Computes shared key ( $K = B^a \bmod p$ )<br>$= 7^5 \bmod 11$                                                                            | 4. Computes shared key $K = A^b \bmod p$<br>$= 10^7 \bmod 11$                                                                                              |
| 5. Stepwise calculation<br>$7^2 = 49 \equiv 5 \bmod 11$<br>$7^4 = 5^2 = 25 \equiv 3 \bmod 11$<br>$7^5 = 3 \times 7 = 21 \equiv 10 \bmod 11$ | 5. Stepwise calculation<br>$10 \equiv -1 \bmod 11$<br>$10^2 = (-1)^2 \equiv 1 \bmod 11$<br>$10^7 = (10^6) \times 10 \equiv 1 \times 10 \equiv 10 \bmod 11$ |
| 6. Final shared key $K = 10$                                                                                                                | 6. Final shared key $K = 10$                                                                                                                               |

# RSA ALGORITHM

1. Choose two different random prime numbers.
2. Calculate  $n = p * q$
3. Calculate  $\phi(n) = (p-1) * (q-1)$
4. Choose 'e' such that  $1 < e < \phi(n)$ . Where, e is coprime to  $\phi(n)$ ,  $\gcd(e, \phi(n)) = 1$
5. Calculate d, such that  $de \equiv 1 \pmod{\phi(n)}$  or d is mul. inv. of e under mod  $\phi(n)$
6. Public key (e,n)      Private key (d,n)

## Encryption

7.  **$C = M^e \pmod n$** , where *M* is the message, *C* is the Cipher text and *e* and *n* are parts of public key.

## Decryption

8.  **$M = C^d \pmod n$** , where *M* is the message and *d* and *n* are parts of private key.

# RSA

**1. Select Prime Numbers :**  $p = 3, q = 11$

**2. Compute**  $n = p \times q = 3 \times 11 = 33$

**3. Compute**  $\phi(n) = (p - 1)(q - 1) = 2 \times 10 = 20$

**4. Choose Public Key**  $e = 3$  such that

- $1 < e < 20$
- $\gcd(e, 20) = 1$

**5. Compute private key**  $d$

- $d \times e \equiv 1 \pmod{20}$  or  $d = e^{-1} \pmod{\phi(n)}$
- $d = 7$

**6. Keys**

- **Public key**  $= (e, n) = (3, 33)$
- **Private key**  $= (d, n) = (7, 33)$

**7. Encryption**

- Let message:

$$M = 4$$

- Encrypt using:

$$C = M^e \pmod{n}$$

$$C = 4^3 \pmod{33} = 64 \pmod{33} = 31$$

- **So , Ciphertext = 31**

**7. Decryption**

- $M = C^d \pmod{n}$

- $M = 31^7 \pmod{33} = 4$

- **Original message recovered = 4**

- 
- Calculator Trick for Multiplicative inverse:  
<https://www.youtube.com/watch?v=gdqENlijflg>